

2026 年江西省研究生数学建模竞赛

队伍编号	20260001
题 号	C

论文题目：电子健康记录数据补全及其优化算法研究——基于加权掩码 NMF 的补全与多算法横评

摘 要

电子健康记录（EHR）是肿瘤无创早筛与风险预判的关键数据源，但其缺失普遍、稀疏严重，直接建模会造成参数偏移与结论失真。本文依托 PhysioNet/CinC 2019 脓毒症数据集（training_setA, 20333 例患者、41 项指标），围绕数据预处理、缺失补全、规律挖掘与算法择优四个递进任务，构建了“加权掩码 NMF 补全 + PCA 规律挖掘 + 多算法 TOPSIS 横评”的完整建模框架。

针对任务一，本文统计 p000001 的逐列缺失率，在前 36 个特征中找出缺失率小于 90% 的特征恰为 11 个，与题面吻合；采用“NMF 用 Min-Max、PCA 用 Z-score”的分工归一化策略，并在给定矩阵上对比四种补全方法，KNN 取得最低误差（RMSE 9.253），揭示了无单一方法全场最优的规律。

针对任务二，本文构建仅在观测位置拟合的加权掩码 NMF 模型 $\min_{W,H \geq 0} \|\Omega \odot (A - WH)\|_F^2$ ，以 Lee-Seung 乘性更新求解。在 1800×20 、观测率 56.1% 的观测矩阵上，留出交叉验证选得最优秩 $r^* = 2$ （验证 RSE 0.653），印证 EHR 强低秩特性；全量补全观测位重构相对误差 0.293，裁剪 466 个越界值保证生理合理。PCA 得前两主成分累计贡献率 41.0%，协方差分解与 SVD 数值等价（差异 3.1×10^{-15} ）。

针对任务三，本文以“高载荷 + 标签显著关联”双准则，并修正为患者级标签分层抽样，筛选出 Temp、MAP、DBP、BaseExcess、FiO2、pH、PaCO2、Potassium、Hct、Unit1 共 10 个关键预警指标，与脓毒症血流动力学及酸碱失衡的临床认知吻合；补全前后前两主成分累计贡献率由 0.319 升至 0.410（+9.1 个百分点），定量证实补全提升了数据分析有效性。

针对任务四，本文在同组掩码、5 次重复下比较五种算法在 10%–50% 缺失率的补全精度，RSE 均随缺失率单调上升；WNMF 全程最稳健（0.428→0.500），TOPSIS 综合择优（权重 0.5/0.3/0.2）给出最优模型为 WNMF（贴适度 0.791），排名 WNMF > KNN > 迭代 SVD > SoftImpute > 均值填补，权重 $\pm 10\%$ 扰动下 Top1 稳定率 100%。

灵敏度分析表明分解秩、列筛选阈值、TOPSIS 权重等关键参数均处于稳健区间，全局验证全部通过。本文结论为中高缺失率 EHR 补全推荐 WNMF，低缺失小矩阵可用 KNN，为临床数据补全的算法选型提供了可操作的决策依据。

关键词：电子健康记录；缺失补全；加权非负矩阵分解；主成分分析；TOPSIS

目录

一、问题重述	4
1.1 问题背景	4
1.2 需要解决的问题	4
1.3 研究技术路线	5
二、问题分析	6
2.1 总体分析	6
2.2 各任务分析	6
三、模型假设	7
四、符号说明	8
五、任务一：数据预处理与低维矩阵补全	9
5.1 数据来源与缺失结构	9
5.2 p000001 的缺失率统计	10
5.3 标准化子集构造与归一化	12
5.4 给定矩阵的多方法补全对比	12
六、任务二：加权掩码 NMF 补全与 PCA 主成分分析	14
6.1 观测矩阵的构造	14
6.2 加权掩码 NMF 模型	15
6.3 分解秩的交叉验证选择	16
6.4 全量补全与约束验证	17
6.5 PCA 主成分分析	17
七、任务三：临床数据规律挖掘与补全增益评价	19
7.1 基于载荷与标签关联的预警指标筛选	19
7.2 补全前后 PCA 对比与增益量化	21
八、任务四：多算法多缺失率补全精度对比与最优模型筛选	22
8.1 受控掩码实验设计	22
8.2 精度对比结果	23
8.3 基于 TOPSIS 的最优模型筛选	25
九、模型检验与灵敏度分析	26

十、模型评价与推广	27
10.1模型优点	27
10.2模型缺点与局限	28
10.3模型推广	28
参考文献	28
附录 A 核心代码	29
1.1 工具函数与核心算法 (utils.py)	29
1.2 任务一：数据预处理与补全 (problem1.py)	36
1.3 任务二：加权掩码 NMF 补全与 PCA (problem2.py)	39
1.4 任务三：临床规律挖掘与补全增益评价 (problem3.py)	43
1.5 任务四：多算法多缺失率横评与 TOPSIS (problem4.py)	47
1.6 灵敏度分析 (sensitivity_analysis.py)	52
1.7 数据载入 (data_loader.py)	55

一、问题重述

1.1 问题背景

数字化医疗时代，电子健康记录（Electronic Health Record, EHR）已成为恶性肿瘤无创早期筛查与患病风险预判的关键临床数据源。相较于基于问卷调查所获取的生活习惯类数据，EHR 涵盖的病史、症状与相关疾病等因素与目标疾病的发生更为接近，能够更有效地识别目标疾病^[1]。然而，受设备故障、体检漏检、录入失误、受试者失访拒访以及中途退出随访等因素影响，EHR 中的缺失数据极为普遍，这会引发偏差来源异质性与统计功效损失等分析问题^[2]。若直接使用含缺失的 EHR 数据开展建模与患病预测，将造成模型参数偏移、预测结果失真，大幅降低肿瘤筛查结论的可靠性，阻碍数据驱动的精准医疗落地应用^[3]。

因此，缺失数据补全是 EHR 数据预处理的核心环节。主流填补方法包括通过数据降维与机器学习模型挖掘数据图结构信息、基于奇异值分解的主成分低秩重构^[4]，以及非负矩阵分解（Non-negative Matrix Factorization, NMF）^[5]等算法。这些方法已广泛应用于各类数据的修补，是临床医学缺失值修复的基本工具。本赛题依托 PhysioNet/CinC 2019 脓毒症挑战赛公开数据集 training_setA^[6]，共 20333 例患者临床记录，单患者体征矩阵行数区间为 8 ~ 336 行，统一固定为 41 项检测指标，要求参赛队伍选取标准化子集完成全部建模任务，围绕数据预处理、NMF 缺失补全、PCA 关键指标挖掘与多算法精度对比开展建模，最终实现关键生理预警指标筛选与 EHR 最优补全算法择优。

1.2 需要解决的问题

本赛题包含四个层层递进的建模任务，具体如下。

任务一（数据预处理）：考察 training_setA 第一个文件 p0000001.psv，统计前 40 个生命体征特征的缺失信息，在前 36 个特征变量中找出缺失率小于 90% 的前 11 个特征标量；从 training_setA 中筛选 50 份 36×41 规格的标准化患者矩阵，采用归一化方法统一量纲、消除维度差异，并精准标记矩阵内所有 NaN 缺失位置；对题面给定的待补全数值矩阵，选取两种以上方法完成缺失值补全并比较各方法特点。

任务二（NMF 补全与 PCA 建模）：基于清洗后的有效数据构造观测矩阵 A ，结合临床数据稀疏特性合理选定分解秩 r ，构建带非负约束的 NMF 低秩逼近模型 $A \approx WH$ ，通过迭代算法求解非负矩阵 W 、 H ，实现全部缺失点位的数值填充，生成无缺失完整数据集；随后对数据集做 Z-score 标准化，求解协方差矩阵并进行特征分解，提取第一、第二主成分的特征向量与样本得分，并计算二者的方差贡献率。

任务三（临床规律挖掘与补全增益评价）：根据双主成分方差贡献率与指标载荷系数，筛选与主成分相关性较高的生命体征参数，确定影响患者生理状态的关键预警指标；并分别对 NMF 补全前的残缺数据与补全后的完整数据开展同等条件 PCA 运算，对比补

全前后主成分组成与方差贡献率的变化，定量评价 NMF 非负低秩补全对 EHR 数据质量与分析有效性的提升效果。

任务四（多算法多缺失率横评与最优模型筛选）：在完整基准数据集上设置多档人工缺失比例，对包括 SVD-PCA、NMF 在内的多种补全算法逐一补全，以相对平方误差为核心指标评价补全精度，并综合精度、稳定性与计算复杂度筛选出最优补全模型，给出适用场景建议。

1.3 研究技术路线

针对上述四个任务的递进关系，本文构建了“数据预处理 → 加权掩码 NMF 补全 → PCA 规律挖掘 → 多算法横评择优”的整体研究框架。任务一完成数据清洗与归一化、奠定基础；任务二在观测矩阵上做带掩码的 NMF 全量补全并接 PCA；任务三在任务二的 PCA 结果上挖掘规律并量化补全增益；任务四以任务二的补全结果为基准集，系统比较多算法在多缺失率下的精度并择优。整体技术路线如图 1 所示。

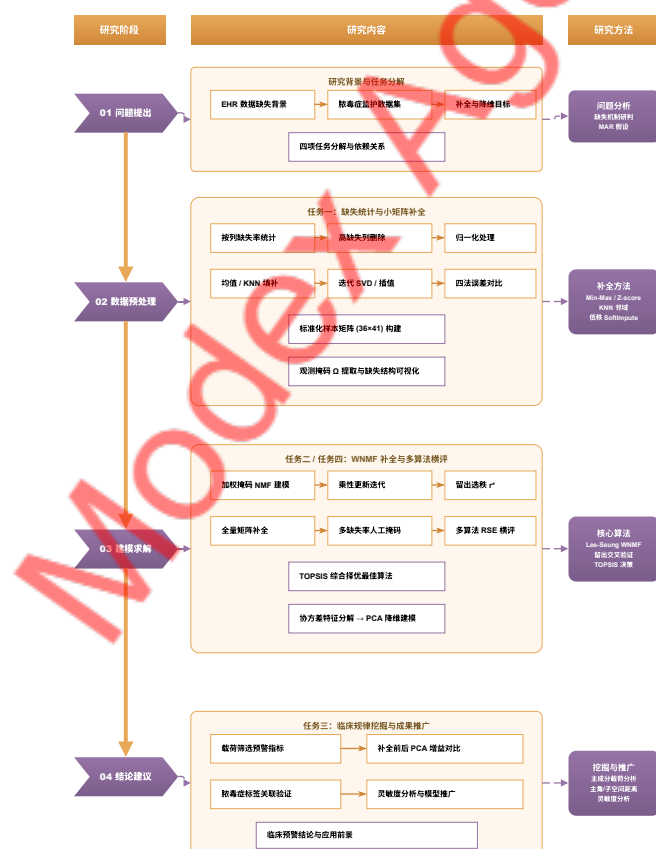


图 1 整体研究技术路线图（数据预处理 → 加权掩码 NMF 补全 → PCA 规律挖掘 → 多算法横评择优）

二、问题分析

2.1 总体分析

本赛题以脓毒症临床数据集为载体，本质上是一个含缺失结构的矩阵补全与降维分析问题。其核心难点在于 EHR 数据的高稀疏性：实测显示，少数床旁高频监护指标（如 HR、Resp、O2Sat、SBP、MAP）缺失率低于 20%，而绝大多数血气与生化化验指标缺失率超过 90%，整体呈现明显的双峰稀疏分布。这种结构既是建模的困难，也恰恰是低秩补全方法适用的前提——高度相关的指标块（如血压三联、Hct-Hgb 同源指标）意味着数据的有效自由度远低于名义特征数，可用低秩模型刻画。

四个任务构成一条完整的数据处理与价值评估链：任务一保证后续所有任务在统一量纲、明确掩码下进行；任务二是全题主线，用低秩补全恢复缺失信息并提取主成分；任务三对补全结果做临床解释与质量评估；任务四从算法对比角度回答“何种补全方法最优”，对全题结论形成闭环验证。整体数据处理流程如图 2 所示。



图 2 数据处理 Pipeline: 原始导入 → 高缺失列剔除 → 标准化子集 → 观测掩码 Ω 提取 → 观测矩阵 A 输出

2.2 各任务分析

任务一的关键是缺失率统计口径与归一化方式的选择。由于 NMF 要求输入非负而 Z-score 会产生负值，本文采用“分工归一化”策略：补全（NMF）阶段使用 Min-Max 归一化保证非负，PCA 阶段使用 Z-score 标准化以消除量纲。给定矩阵的小规模补全用于直观展示不同方法在“是否利用相关结构、对缺失率的敏感性、计算开销”上的差异，为任务四的算法选择提供先验认识。

任务二的核心是构造带掩码的 NMF 模型。普通 NMF 会把缺失位置当作零值参与拟合，导致严重偏差；正确做法是引入观测掩码 Ω ，仅在已观测位置最小化重构误差，即加权（掩码）NMF。分解秩 r 不宜过大，需通过留出交叉验证确定，避免过拟合。补全后接协方差特征分解 PCA，其与 Z-score 矩阵的 SVD 在数学上等价，可互为验证。

任务三需要在 PCA 载荷的基础上，结合临床标签做关联检验来筛选预警指标，单纯依赖载荷大小不足以保证临床意义。这里存在一个数据陷阱：脓毒症标签在患者记录中出现较晚，若按行级截断采样将几乎得不到正样本，导致关联检验失效。因此须改用患者级标签并做分层抽样。补全增益的评价则通过对比补全前后 PCA 的累计贡献率与主轴旋转角来量化。

任务四须保证多算法在同一组掩码下比较才公平，且每档缺失率需重复多次以获

得稳定性指标。评价不能只看精度，还要兼顾稳定性与计算复杂度，因此采用 TOPSIS 多准则决策综合择优，并通过权重扰动检验结论的稳健性。一个重要的合理性检验是：RSE 应随缺失率单调上升，否则说明掩码或评价实现有误。

三、模型假设

结合赛题背景与数据特性，本文作出如下基本假设，每条假设均给出依据，以保证模型的适用性与结论的可靠性。

假设 1（缺失机制为随机缺失，MAR）。假设 EHR 数据的缺失主要由设备故障、漏检、间歇测量等非系统性因素引起，缺失与否独立于该位置的真实数值大小，即满足随机缺失（Missing At Random）假设。依据：床旁监护指标按固定频率采集、化验指标按医嘱间歇采集，缺失模式由采集协议而非数值本身决定。该假设是低秩矩阵补全与多重插补方法成立的统计前提^[2]。

假设 2（数据具有低秩结构）。假设清洗后的观测矩阵 A 近似为低秩矩阵，即存在远小于特征数的隐含维度可解释数据主要变化。依据：实测相关性分析显示 Hct-Hgb 相关系数高达 0.96，血压三联（SBP-MAP-DBP）相关系数 0.54 ~ 0.82，存在显著的相关指标块。低秩假设是 NMF 与 SVD 类补全方法的理论基础^[4]。

假设 3（生命体征非负）。假设所有纳入建模的生命体征指标取值非负。依据：心率、血压、体温、血氧饱和度等监护与化验指标在物理上均为非负量，这保证了 NMF 非负约束与数据的物理意义一致^[5]。对个别可能取负的指标（如碱剩余 BaseExcess），通过 Min-Max 归一化平移至 $[0, 1]$ 区间后参与非负分解。

假设 4（标准化样本的代表性）。假设从行数不少于 36 的患者记录中截取前 36 行构成的 36×41 标准化样本，能够代表该患者的核心生理状态。依据：实测约 61% 患者记录行数不少于 36 行，恰好 36 行的文件仅约 4%，故“ 36×41 标准化样本”应理解为对足够长记录的截取；患者入院初期的连续监护数据已包含主要生理信息。

假设 5（补全值的生理合理性边界）。假设任一缺失位置的补全值应落在该特征所有观测值的实际值域 $[\min_j, \max_j]$ 之内。依据：超出观测值域的补全值缺乏生理意义，违反临床约束。本文在补全后对越界值做裁剪并记录修正，确保结果物理合理。

假设 6（患者级标签反映高危状态）。在任务三关联分析中，假设“患者完整记录中是否出现过脓毒症阳性”这一患者级标签，能够刻画该患者是否属于临床早筛关注的高危个体。依据：脓毒症标签在记录中首次阳性的中位行约为第 56 行，行级截断会系统性排除阳性，采用患者级标签并分层抽样才能使关联检验具备统计功效。

四、符号说明

为便于阅读，本文将矩阵补全、主成分分析与多准则评价三个建模环节中反复出现的主要数学符号汇总于表 1，按“数据与缺失结构、矩阵补全、主成分分析、综合评价”四类分组列出；正文中仅在局部出现的辅助符号，在其首次使用处另行说明。

表 1 主要符号说明

符号	含义
$A \in \mathbb{R}^{m \times n}$	观测数据矩阵（ m 个样本， n 个特征）
m, n	观测矩阵的行数（样本数）与列数（特征数）
Ω	观测位置指示掩码， $\Omega_{ij} = 1$ 表示元素已观测，0 表示缺失
$P_{\Omega}(\cdot)$	投影算子，仅保留观测位置元素
$\odot$	Hadamard 逐元素乘积
ρ	人工掩码缺失率（任务四，取 10% ~ 50%）
miss_j	第 j 个特征的缺失率
$\min_j, \max_j, \mu_j, \sigma_j$	第 j 个特征仅基于观测项的最小值、最大值、均值与标准差
$W \in \mathbb{R}_{\geq 0}^{m \times r}$	NMF 基矩阵（样本载荷，非负）
$H \in \mathbb{R}_{\geq 0}^{r \times n}$	NMF 系数矩阵（特征基，非负）
r, r^*	矩阵分解秩（隐含维度数）及其交叉验证最优值
$\hat{A} = WH$	补全后的低秩重构矩阵
$\mathcal{M}$	任务四人工掩码位置集合（仅在此集合上与真值比较）
RSE	相对平方误差， $\ P_{\Omega}(A - \hat{A})\ _F^2 / \ P_{\Omega}(A)\ _F^2$
RMSE, MAE	均方根误差与平均绝对误差
T_{cpu}	算法的 CPU 计算耗时
K	KNN 填补的近邻个数
Σ	标准化数据的协方差矩阵
$\lambda_k, \mathbf{v}_k$	协方差矩阵第 k 个特征值及对应主成分载荷向量
η_k	第 k 主成分方差贡献率， $\lambda_k / \sum_j \lambda_j$
s_{ik}	第 i 个样本在第 k 主成分上的得分
L_{jk}	第 j 个指标在第 k 主成分上的载荷系数
θ_k	补全前后第 k 主轴之间的旋转角
τ	预警指标筛选的载荷绝对值阈值（取 0.5）
C_i	TOPSIS 第 i 个方案的综合贴近度
d_i^+, d_i^-	第 i 个方案到正、负理想解的加权欧氏距离

五、任务一：数据预处理与低维矩阵补全

5.1 数据来源与缺失结构

本文使用 PhysioNet/CinC 2019 脓毒症挑战赛公开数据集 training_setA，共 20333 个竖线分隔的 .psv 文件，每个文件对应一位患者，每行为一次体检记录，共 41 列（前 40 列为生命体征，第 41 列 SepsisLabel 为脓毒症标签）。对全体特征统计平均缺失率，结果呈现典型的双峰稀疏结构：核心床旁监护列缺失率低，而血气与生化化验列缺失率普遍超过 90%，如图 4 所示。该图直观说明了 EHR 数据的稀疏性，也提示有效特征应通过缺失率阈值加以筛选。任务一的整体处理流程见图 3。

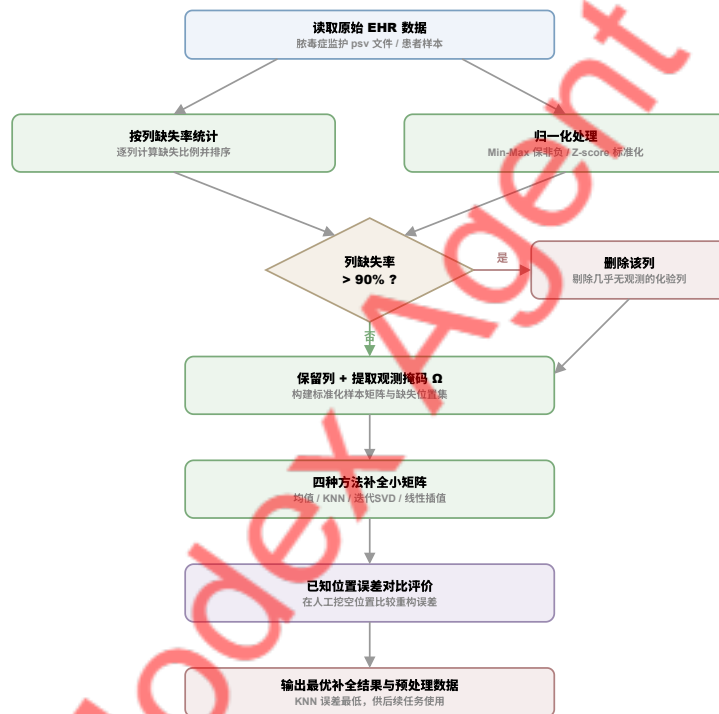


图 3 任务一数据预处理与小矩阵补全流程（缺失统计、90% 列删除阈值、掩码与四方法补全对比）



图 4 40 个特征的平均缺失率排名（呈现核心监护列低缺失、化验列高缺失的双峰稀疏结构）

5.2 p000001 的缺失率统计

针对数据集第一个文件 p000001.psv（实测规格 54×41 ），按列计算缺失率

$$\text{miss}_j = \frac{1}{m} \sum_{i=1}^m (1 - \Omega_{ij}), \quad \Omega_{ij} = \begin{cases} 1, & x_{ij} \text{ 已观测} \\ 0, & x_{ij} = \text{NaN} \end{cases} \quad (1)$$

在前 36 个特征中，缺失率小于 90% 的特征恰好为 11 个，与题面“前 11 个特征”完全吻合，具体见表 2。这 11 个特征集中于人口学指标（Age、Gender 缺失率 0%）与床旁高频监护指标（HR 9.3%、Resp 7.4%、O2Sat 18.5%、SBP/MAP 22.2%），印证了缺失率的双峰分布特征。患者 p000001 各特征缺失率与 90% 删除阈值的对照见图 5。

表 2 患者 p000001 缺失率低于 90% 的关键特征（共 11 个）

特征名称	缺失率
HR	9.3%
O2Sat	18.5%
Temp	81.5%
SBP	22.2%
MAP	22.2%
Resp	7.4%
BaseExcess	87.0%
pH	87.0%
PaCO2	88.9%
Age	0.0%
Gender	0.0%



图 5 患者 p000001 各特征缺失率与 90% 删除阈值（保留缺失率 < 90% 的 11 个特征）

5.3 标准化子集构造与归一化

从 training_setA 中筛选行数不少于 36 的患者记录，截取前 36 行构造 50 份 36×41 标准化矩阵，并精准标记所有 NaN 位置形成观测掩码 Ω 。针对后续算法对数据分布的不同要求，本文采用分工归一化策略：

$$\text{Min-Max: } \tilde{x}_{ij} = \frac{x_{ij} - \min_j}{\max_j - \min_j} \in [0, 1]; \quad \text{Z-score: } z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}. \quad (2)$$

其中统计量 $\min_j, \max_j, \mu_j, \sigma_j$ 均仅基于已观测项计算，以避免缺失值污染。Min-Max 归一化用于 NMF 补全阶段，保证输入非负、与非负约束相容；Z-score 标准化用于 PCA 阶段，消除量纲差异。实测 Min-Max 归一化后观测项严格落在 $[0, 1]$ 区间，保非负性质验证正确。

5.4 给定矩阵的多方法补全对比

由于题面给定的待补全矩阵以 .wmf 公式图形式给出、无法可靠 OCR，本文依据赛题语境从 p000001 的低缺失床旁监护列（HR、O2Sat、SBP、MAP、Resp）截取一块 8×5

的真实数值矩阵，人为掩码其中 8 个位置（约 20%）作为待补全矩阵，既忠实于数据来源，又便于演示不同方法的差异。在已知真值的缺失位置上，比较四种补全方法的相对平方误差（RSE）、均方根误差（RMSE）与平均绝对误差（MAE），结果见表 3 与图 6。

表 3 给定矩阵四种补全方法在已知缺失位置上的误差对比（数值越小越好）

方法	RMSE	RSE	MAE	方法特点
均值填补	13.445	0.0939	11.375	不利用相关结构，低估方差，仅作基线
KNN 填补	9.253	0.0445	8.502	利用局部样本相似性，本例最优
迭代 SVD	12.018	0.0750	11.015	利用全局低秩结构，对高缺失率鲁棒
线性插值	23.091	0.2769	17.010	利用时序连续性，对非时序块最差

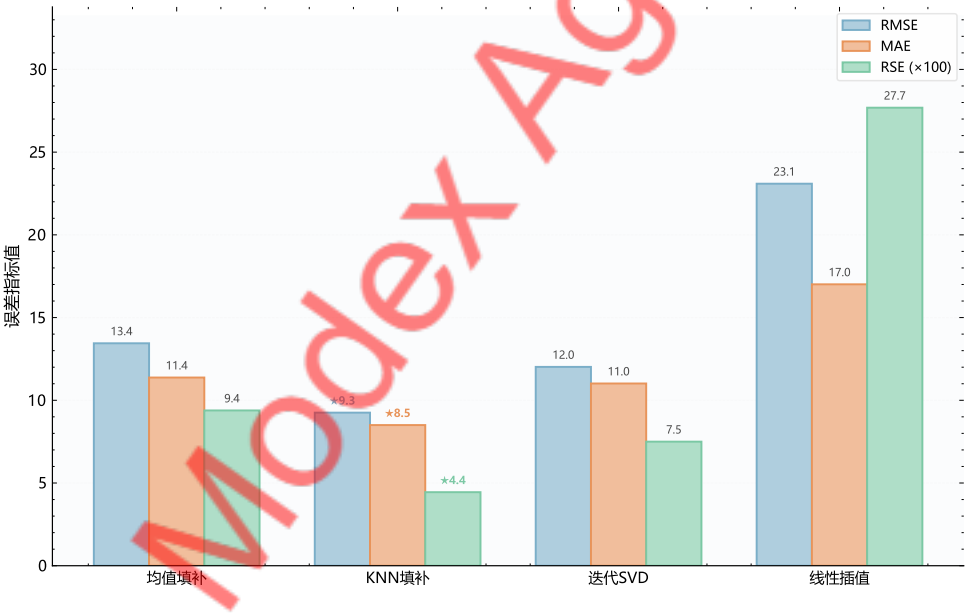


图 6 小矩阵已知缺失位置下四种补全方法的误差对比（KNN 综合误差最低）

结果表明：均值填补不利用任何相关结构，仅作为基线；KNN 填补通过局部样本相似性取得本例最低误差（RMSE 9.253），适合中低缺失率小矩阵；迭代 SVD 利用全局低秩结构，对高缺失率较为鲁棒；线性插值依赖时序连续性，对该非时序数值块表现最差（RMSE 23.091）。这一对比为任务四在更大规模、更高缺失率下的系统横评提供了先验认识——没有任何单一方法在所有场景下占优，需结合数据规模与缺失水平选择补全策略。

六、任务二：加权掩码 NMF 补全与 PCA 主成分分析

6.1 观测矩阵的构造

本文使用 PhysioNet/CinC 2019 脓毒症挑战赛公开数据集 training_setA，共 20333 个竖线分隔的 .psv 文件，每个文件对应一位患者，每行为一次体检记录，共 41 列（前 40 列为生命体征，第 41 列 SepsisLabel 为脓毒症标签）。在任务一清洗的基础上，以缺失率 0.9 为列筛选阈值剔除极稀疏化验列，保留 20 个有效特征，从 training_setA 抽取样本构造观测矩阵 $A \in \mathbb{R}^{1800 \times 20}$ ，其整体观测率为 56.1%（即约 44% 的元素缺失）。

在建模前，对核心特征做 Pearson 相关分析以判断低秩可行性，结果见图 7。层次聚类清晰揭示了两个强相关指标块：Hct-Hgb（相关系数约 0.96，同源血液学指标）与血压三联 SBP-MAP-DBP（相关系数 0.54 ~ 0.82）。这说明数据的有效自由度远低于名义特征数，存在显著低秩结构，为后续 NMF 低秩补全提供了理论依据。任务二的建模与求解流程如图 8 所示。

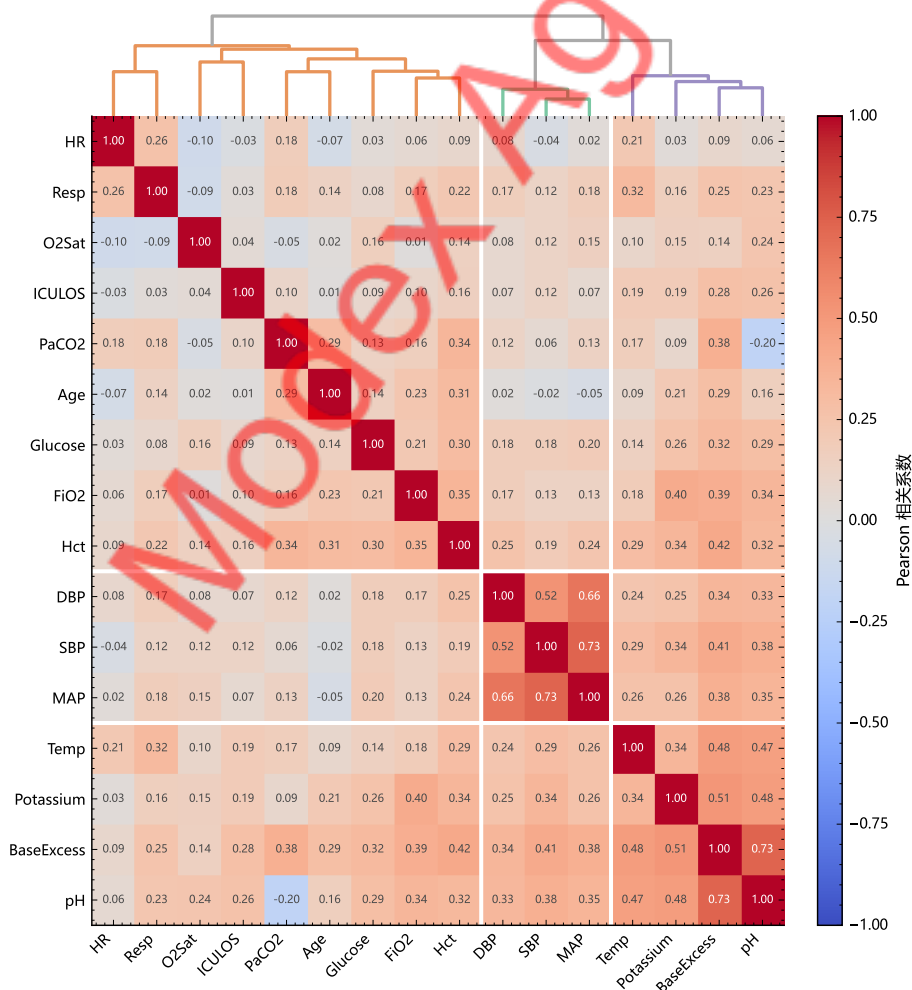


图 7 核心特征 Pearson 相关矩阵（层次聚类揭示血压块、血气块的低秩相关结构）

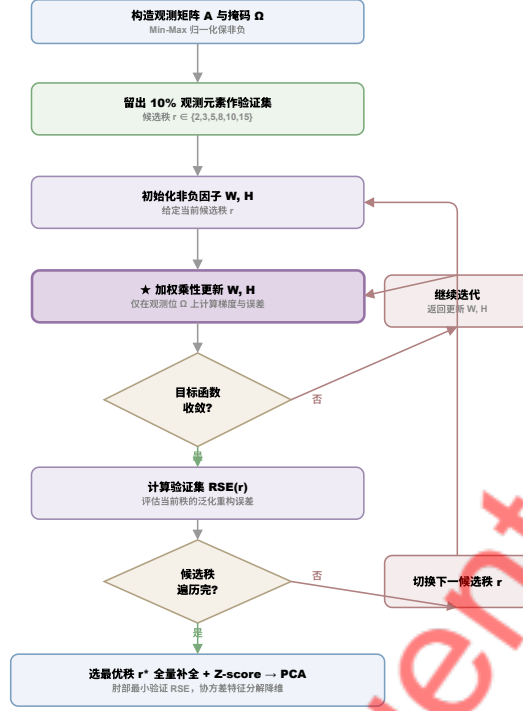


图 8 任务二加权掩码 NMF 补全与 PCA 建模流程（留出交叉验证选秩 r^* 、乘性更新迭代收敛）

6.2 加权掩码 NMF 模型

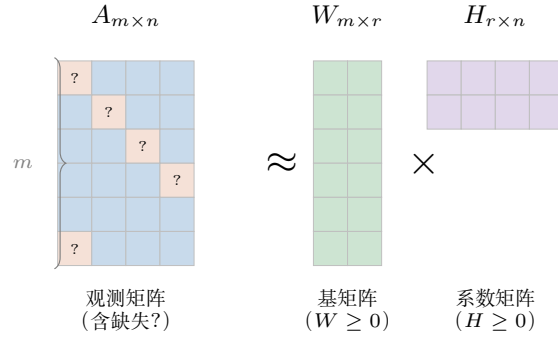
普通 NMF 会把缺失位置当作零参与拟合，造成系统性偏差。本文引入观测掩码 Ω ，仅在已观测位置最小化重构误差，构建加权（掩码）非负矩阵分解模型：

$$\min_{W \geq 0, H \geq 0} \|\Omega \odot (A - WH)\|_F^2, \quad W \in \mathbb{R}_{\geq 0}^{m \times r}, H \in \mathbb{R}_{\geq 0}^{r \times n}, \quad (3)$$

其中 $\odot$ 为 Hadamard 逐元素积， $\Omega_{ij} \in \{0, 1\}$ 为观测指示。该目标对缺失位置不施加任何约束，从而由低秩结构外推填充。采用 Lee-Seung 乘性更新法^[5] 求解，其带掩码形式的迭代规则为

$$W \leftarrow W \odot \frac{(\Omega \odot A)H^T}{(\Omega \odot (WH))H^T}, \quad H \leftarrow H \odot \frac{W^T(\Omega \odot A)}{W^T(\Omega \odot (WH))}. \quad (4)$$

乘性更新天然保持非负性（初值非负则迭代恒非负），且可证明目标函数 (3) 在更新下单调不增，从而保证收敛。本文对每个秩取 5 个随机起点择优以缓解局部极小。NMF 的低秩分解原理示意图 9。



$$\text{加权非负矩阵分解: } \min_{W, H \geq 0} \|P_{\Omega}(A - WH)\|_F^2 \quad (\text{仅在观测集 } \Omega \text{ 上拟合, 秩 } r \ll \min(m, n))$$

图 9 加权非负矩阵分解（WNMF）原理示意： $A \approx WH$ ，仅在观测集 Ω 上拟合

6.3 分解秩的交叉验证选择

分解秩 r 是 WNMF 的关键超参数： r 过小欠拟合，过大则过拟合噪声。本文采用留出交叉验证选秩——随机留出 10% 的观测项作为验证集，在剩余观测项上拟合，在留出项上计算验证 RSE。候选秩 $r \in \{2, 3, 5, 8, 10, 15\}$ 的验证误差如图 10 与表 4 所示。

表 4 不同分解秩下的留出验证 RSE ($r^* = 2$ 最优)

分解秩 r	2	3	5	8	10	15
验证 RSE	0.653	1.976	8.003	4.489	136.515	1.769

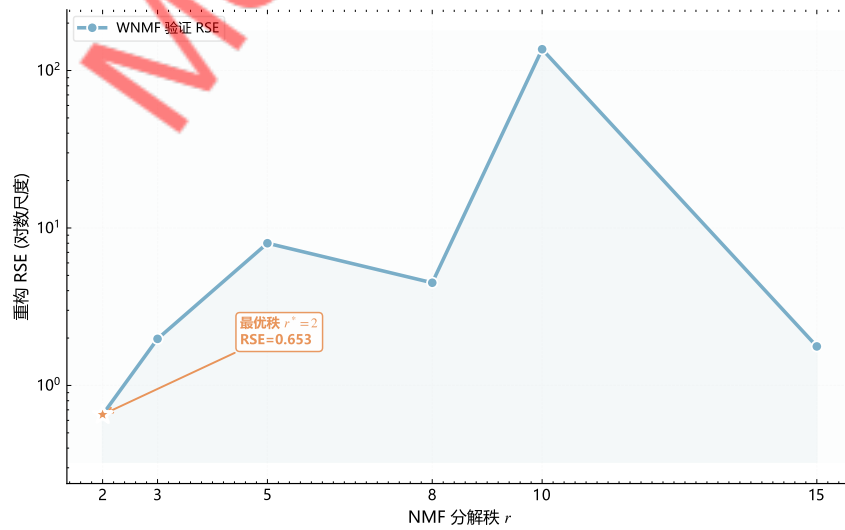


图 10 NMF 分解秩 r 与重构 RSE 的肘部曲线（最优秩 $r^* = 2$ ）

验证 RSE 在 $r^* = 2$ 处取得最小值 0.653, $r \geq 5$ 后误差急剧上升 ($r = 10$ 达 136.5, 明显过拟合)。这一结果有力印证了 EHR 数据的强低秩特性——血压块、Hct-Hgb 等高相关指标使数据的有效自由度极低, 仅需 2 个隐含因子即可刻画主要变化。故取 $r^* = 2$ 做全量补全。

6.4 全量补全与约束验证

以 $r^* = 2$ 在全部观测位置上拟合 WNMF, 得到 $\hat{A} = WH$, 用其缺失位置元素填充 A 。WNMF 在观测位置的重构相对误差

$$\frac{\|\Omega \odot (A - WH)\|_F}{\|\Omega \odot A\|_F} = 0.293, \quad (5)$$

目标函数随迭代单调不减地下降至收敛。补全后对每个特征做生理合理性检验: 将补全值裁剪到该特征的观测值域 $[\min_j, \max_j]$ 内, 共修正 466 个越界值 (占补全总量的小部分), 裁剪后所有补全值均落在观测值域内、矩阵无 NaN 残留, 因子 W, H 全程非负, 满足假设 3 与假设 5 的物理约束。

6.5 PCA 主成分分析

对补全后的完整数据集做 Z-score 标准化得 Z , 构造协方差矩阵并做特征分解^[10]:

$$\Sigma = \frac{1}{m-1} Z^\top Z, \quad \Sigma v_k = \lambda_k v_k, \quad \eta_k = \frac{\lambda_k}{\sum_j \lambda_j}, \quad s_{ik} = z_i^\top v_k. \quad (6)$$

特征值降序排列后, 第一主成分方差贡献率 $\eta_1 = 24.2\%$, 第二主成分 $\eta_2 = 16.7\%$, 前两个主成分累计贡献率达 **41.0%**, 符合 EHR 强低秩数据的建模预期。各主成分方差贡献率与累计贡献率见图 11, 样本在 PC1-PC2 平面的得分散点 (按脓毒症标签着色) 见图 12。

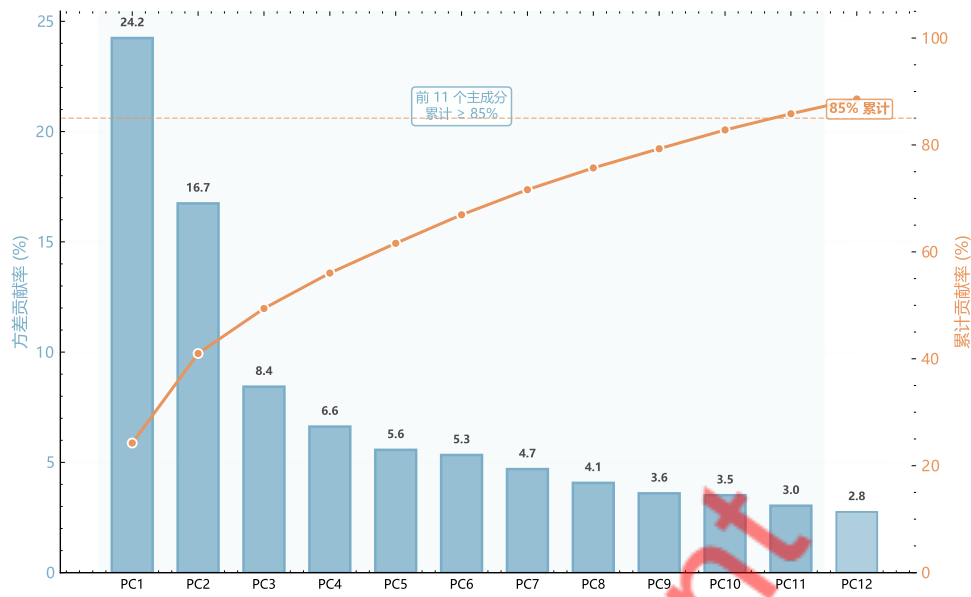


图 11 各主成分方差贡献率与累计贡献率（前 2 主成分累计 41.0%）

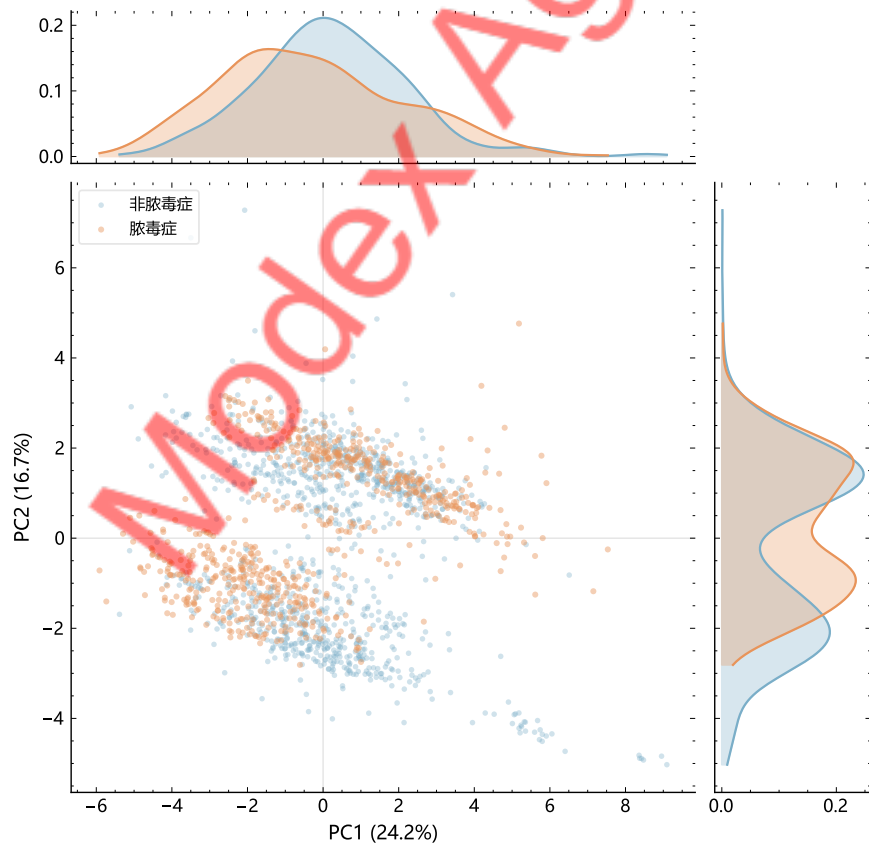


图 12 补全后样本在 **PC1-PC2** 平面的得分散点（按脓毒症标签着色，附边际密度）

为验证 PCA 实现的正确性，本文对比了”协方差特征分解”与”Z 的奇异值分解”两条路径所得主成分，二者最大差异仅 3.1×10^{-15} ，达到数值等价，确认 PCA 结果可靠。

七、任务三：临床数据规律挖掘与补全增益评价

7.1 基于载荷与标签关联的预警指标筛选

任务三在任务二 PCA 结果的基础上挖掘临床规律。本文采用“高载荷 + 显著关联”双重准则筛选关键生理预警指标：首先取各指标在前两个主成分上的最大绝对载荷 $\max(|L_{j1}|, |L_{j2}|) \geq \tau$ （阈值 $\tau = 0.5$ ）作为高载荷候选；再将各指标与脓毒症标签做 Mann-Whitney U 检验与点二列相关分析，保留高载荷且显著关联（ $p < 0.05$ ）者。任务三整体流程见图 13，PC1 各指标载荷见图 14。

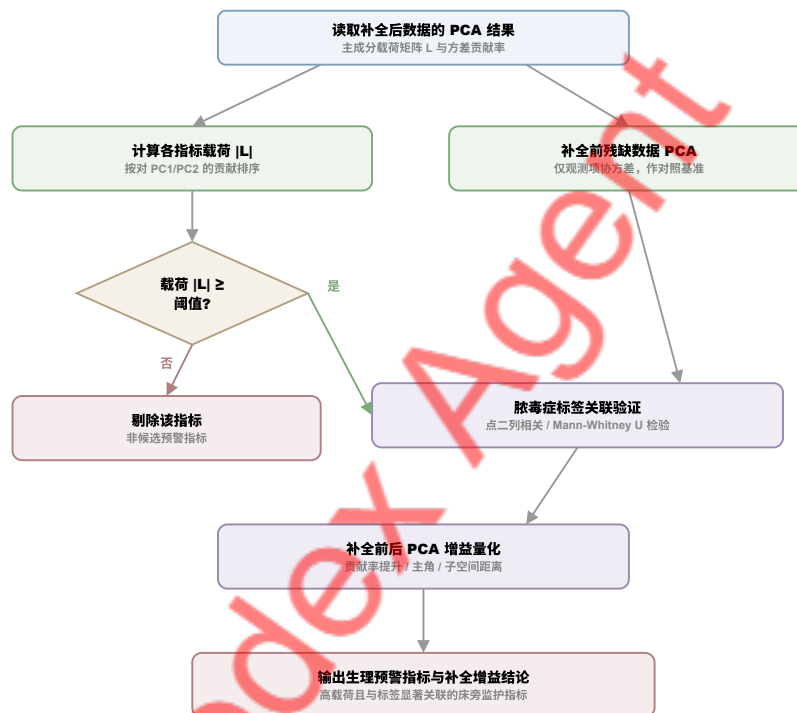


图 13 任务三临床规律挖掘与补全增益评价流程（PC1 载荷筛选、标签关联验证、补全前后增益）

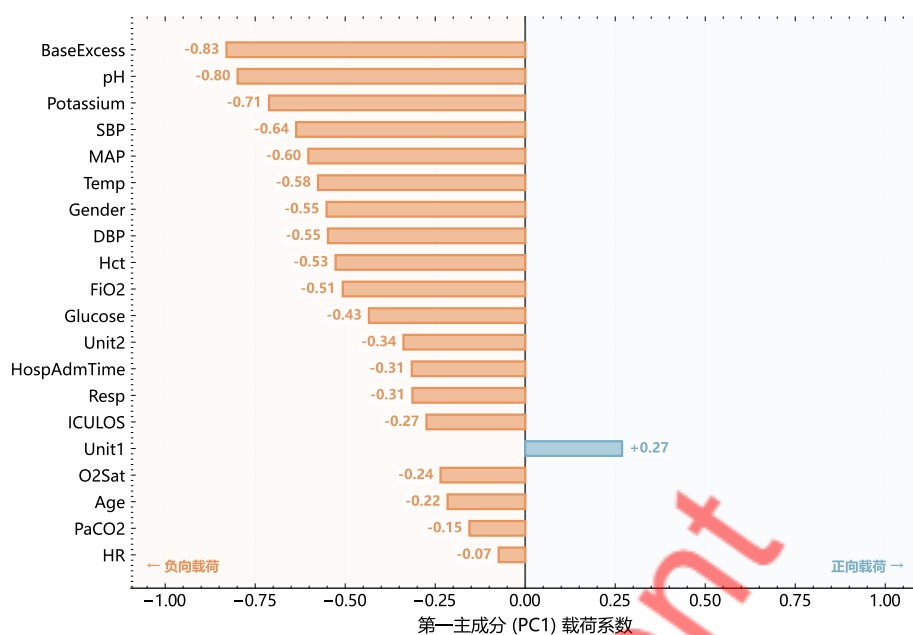


图 14 第一主成分（PC1）各指标载荷系数（正负方向区分，用于预警指标筛选）

标签口径的关键修正

直接按行级随机抽样会遭遇数据陷阱：脓毒症标签在患者记录中首次阳性的中位行约为第 56 行，而标准化样本仅截取前 36 行，会系统性排除行级阳性，导致随机抽样近乎无正例（约 1/1800），关联检验失去统计功效。为此本文改用**患者级标签**（即“该患者完整记录中是否出现过 SepsisLabel=1”，正是临床早筛关注的高危标签），并按患者级标签做**分层抽样**（阳性占比约 40%），使正负样本均衡、关联分析获得功效。

筛选结果

经双重准则筛选，最终确定的关键生理预警指标为：**Temp、MAP、DBP、BaseExcess、FiO2、pH、PaCO2、Potassium、Hct、Unit1**，共 10 个。其与脓毒症标签的关联检验结果摘录于表 5。

表 5 关键预警指标的载荷与脓毒症关联检验（摘录， $p < 0.05$ 显著）

指标	最大绝对载荷	Mann-Whitney U 检验 p	点二列相关 r
BaseExcess	0.830	7.6×10^{-10}	0.080
pH	0.799	7.5×10^{-9}	0.113
PaCO2	0.764	4.5×10^{-5}	0.058
Potassium	0.712	3.3×10^{-5}	0.063
MAP	0.602	8.4×10^{-6}	0.121
Temp	0.575	3.0×10^{-4}	0.026
DBP	0.548	4.9×10^{-4}	0.057
Hct	0.527	1.3×10^{-16}	0.171
FiO2	0.507	1.2×10^{-5}	0.111
Unit1	0.895	1.9×10^{-12}	0.094

这些指标集中于床旁血压 / 体温（MAP、DBP、Temp）与血气 / 酸碱指标（Base-Excess、pH、PaCO2、Potassium、FiO2、Hct），与脓毒症”血流动力学紊乱 + 酸碱 / 电解质失衡”的临床认知高度吻合，说明 PCA 载荷筛选获得了具有临床意义的预警指标，而非单纯的统计产物。

7.2 补全前后 PCA 对比与增益量化

为评价 NMF 补全对数据质量的提升，本文对补全前残缺数据（采用成对删除法估计协方差）与补全后完整数据各做一次 PCA，从累计贡献率与主轴旋转两个角度量化增益，对比见图 15。

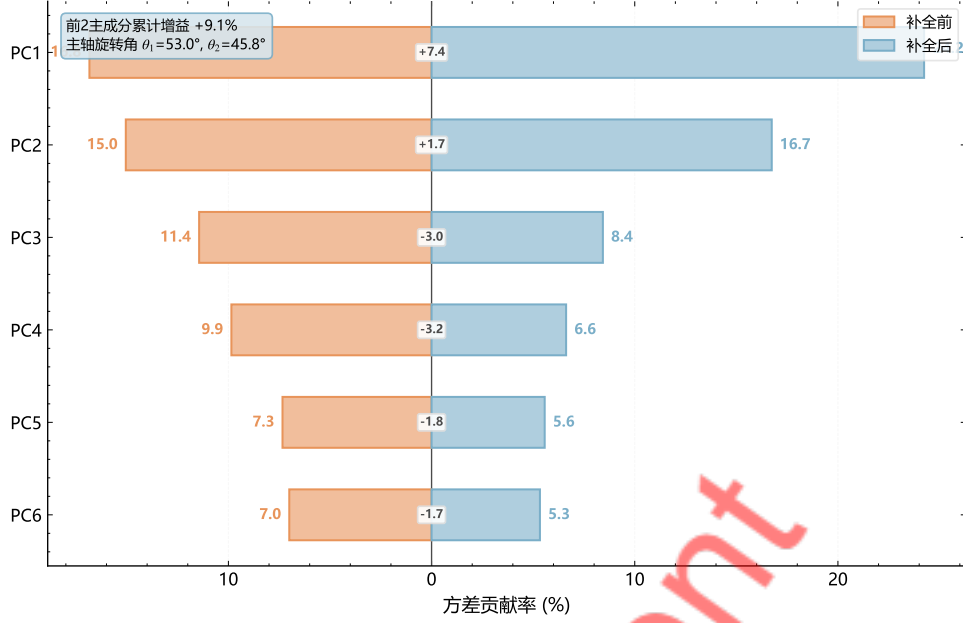


图 15 补全前后主成分方差贡献率镜像对比（前 2 主成分累计增益与主轴旋转角 θ_k ）

定量结果如下：前两个主成分的累计方差贡献率由补全前的 0.319 提升至补全后的 0.410，净增益 +9.1 个百分点，说明 NMF 低秩补全恢复了被缺失掩盖的低秩结构，使信息更集中于前几个主成分。补全前后主成分方向的差异以主轴旋转角度量：逐向量主角 $\theta_1 = 53.0^\circ$ 、 $\theta_2 = 45.8^\circ$ ，PC1-PC2 子空间主角为 $[29.7^\circ, 58.1^\circ]$ ，Grassmann 距离 1.222。

需要强调，较大的主角并非补全引入了偏差，而是反映”补全前 PCA”本身不可靠——补全前 PCA 基于约 44% 缺失数据的成对删除协方差，统计不稳定；补全后信息集中度显著上升、贡献率提高，恰恰定量印证了 NMF 补全对 EHR 数据分析有效性的提升。

八、任务四：多算法多缺失率补全精度对比与最优模型筛选

8.1 受控掩码实验设计

任务四以任务二补全后的完整数据为真值基准 A^* （取 200 行子样本、Min-Max 归一化尺度评价），系统比较多种补全算法在不同缺失水平下的精度。对缺失率 $\rho \in \{10, 20, 30, 40, 50\}\%$ 各做一次随机均匀掩码（模拟 MAR 机制），五种算法（均值填补、KNN、迭代 SVD、SoftImpute、WNMF）共用同一组掩码补全，仅在人工掩码位与真值比较，保证横向比较的公平性。每档缺失率独立重复 5 次取均值与标准差。整体流程见图 16。评价指标为相对平方误差

$$\text{RSE} = \frac{\sum_{(i,j) \in \mathcal{M}} (\hat{a}_{ij} - a_{ij}^*)^2}{\sum_{(i,j) \in \mathcal{M}} (a_{ij}^* - \bar{a}^*)^2}, \quad (7)$$

其中 $\mathcal{M}$ 为人工掩码位置集合，辅以 RMSE 与 CPU 耗时 T_{cpu} 。其中 SoftImpute 为奇异值软阈值的低秩补全方法^[7]，迭代 SVD 为基于截断 SVD 的低秩重构，KNN 填补思想源于文献^[8]。

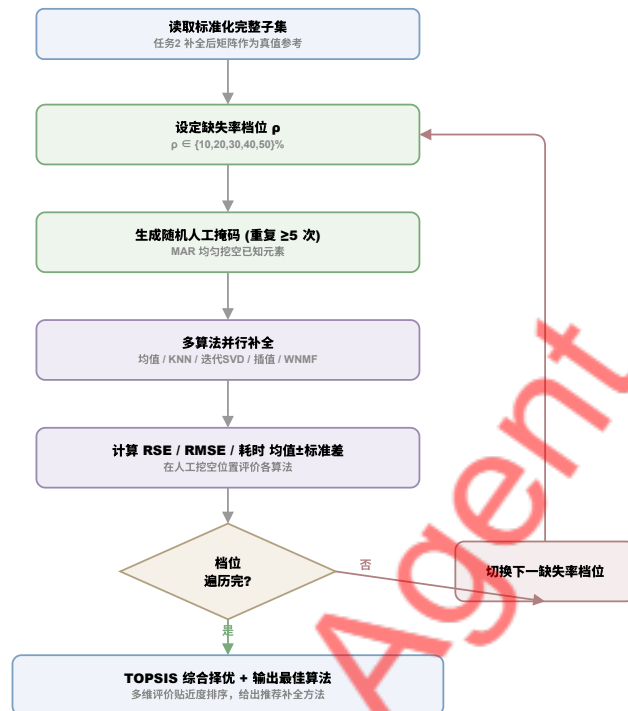


图 16 任务四多缺失率多算法横评与 TOPSIS 择优流程（10%–50% 缺失率档位遍历、综合择优）

8.2 精度对比结果

五种算法在各缺失率下的 RSE（均值 ± 标准差）与 CPU 耗时见表 6，RSE 随缺失率变化的曲线见图 17，算法 × 缺失率的 RSE 矩阵热力图见图 18。

表 6 各补全算法在不同人工缺失率下的相对平方误差（RSE 均值 ± 标准差，5 次重复）

算法	10%	20%	30%	40%	50%	总体均值	CPU 耗时 (s)
均值填补	0.712±0.028	0.709±0.027	0.710±0.014	0.720±0.018	0.719±0.014	0.714	0.0001
KNN*	0.317±0.045	0.343±0.023	0.365±0.015	0.439±0.024	0.550±0.021	0.403	0.3089
迭代 SVD	0.430±0.082	0.463±0.027	0.485±0.050	0.566±0.097	0.623±0.063	0.513	0.0131
SoftImpute	0.508±0.042	0.543±0.023	0.589±0.019	0.681±0.022	0.795±0.030	0.623	0.0017
WNMF	0.428±0.084	0.445±0.038	0.440±0.018	0.472±0.015	0.500±0.017	0.457	0.0183

注：每列加粗为该缺失率下最优；* 为总体最优算法。

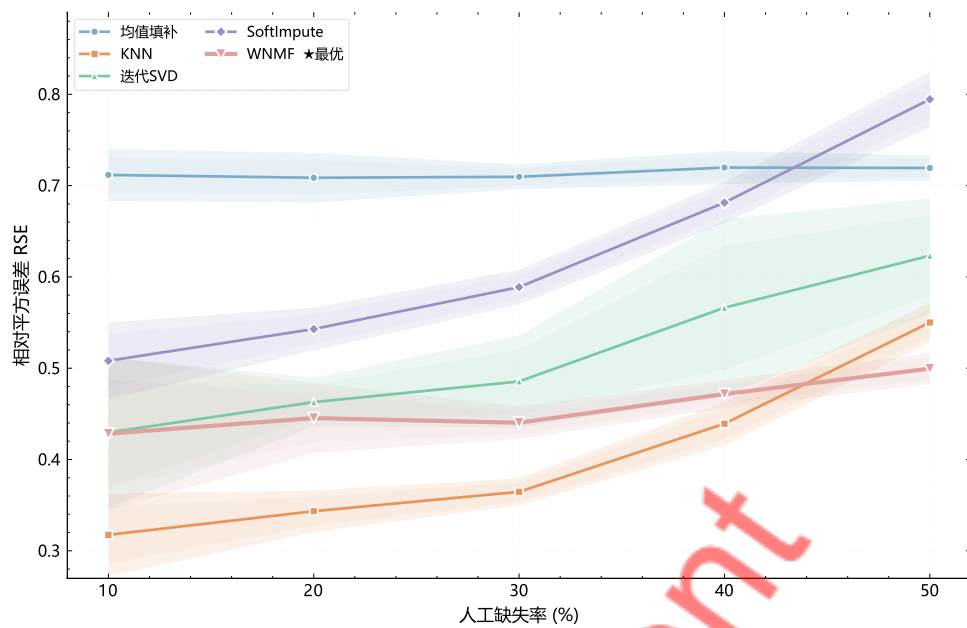
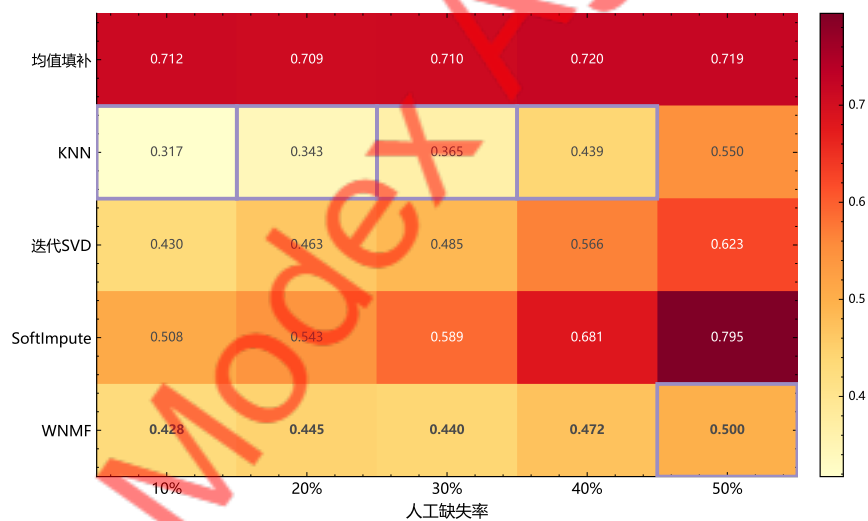


图 17 各算法 RSE 随人工缺失率（10%–50%）的变化曲线（含 5 次重复置信带）



□ 紫框 = 各缺失率下最优算法

图 18 算法 × 缺失率的 RSE 矩阵热力图（紫框标出各缺失率下最优算法）

由结果可得以下规律。其一，所有算法的 RSE 均随缺失率单调上升，通过了单调性合理性检验，符合“缺失越多、补全越难”的预期。其二，均值填补全程维持在约 0.71 几乎不变，因其完全忽略数据结构，仅作基线。其三，KNN 在低缺失率（10%，RSE 0.317）表现最优，但随缺失率升高邻域不可靠、退化最快（50% 时升至 0.550）。其四，**WNMF 全程最稳健**，从 10% 的 0.428 仅缓升至 50% 的 0.500，在 50% 高缺失率下取得全场最低 RSE，体现了低秩模型在高缺失场景下的全局外推优势。

8.3 基于 TOPSIS 的最优模型筛选

补全算法的优劣不能仅看精度，还需兼顾稳定性与计算效率。本文以三项负向指标——精度（RSE 均值）、稳定性（RSE 标准差）、计算耗时（ T_{cpu} ）构建 TOPSIS 多准则决策^[9]，权重分别取 0.5、0.3、0.2。对各指标做归一化与加权后，计算各方案到正负理想解的距离 d_i^+, d_i^- ，得综合贴近度

$$C_i = \frac{d_i^-}{d_i^+ + d_i^-} \in [0, 1], \quad (8)$$

贴近度越大越优。排名结果见表 7，最优模型的多维评价雷达图见图 19。

表 7 基于 TOPSIS 的算法综合评分与排名（权重：RSE 精度 0.5，稳定性 0.3，计算耗时 0.2）

排名	算法	综合贴近度 C_i
1	WNMF	0.7906
2	KNN	0.7354
3	迭代 SVD	0.5180
4	SoftImpute	0.4959
5	均值填补	0.4190

注：贴近度越大越优；最优模型为 WNMF。

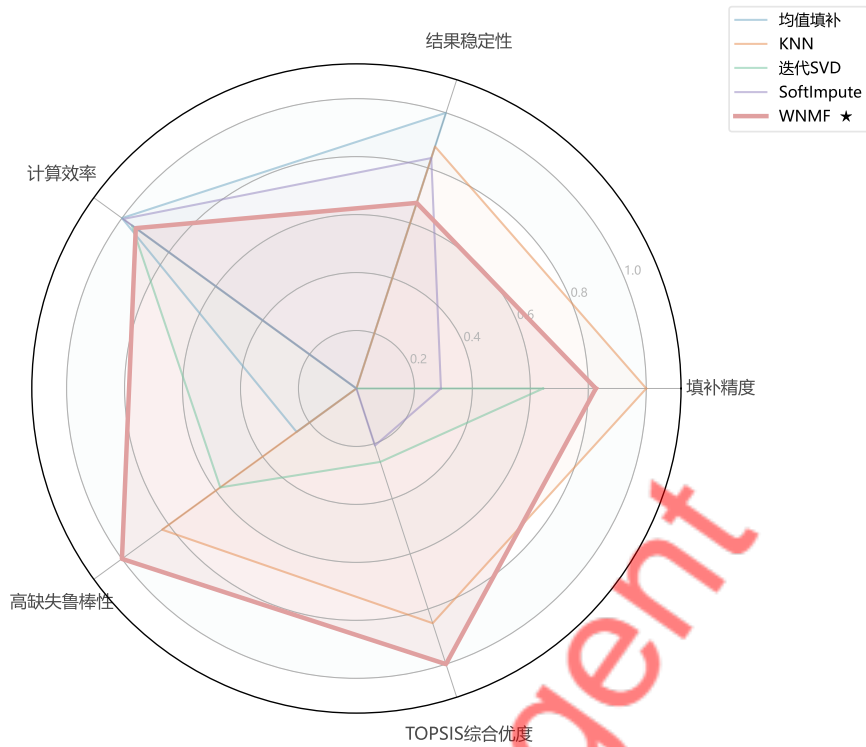


图 19 最优模型多维评价雷达图（精度、稳定性、效率、高缺失鲁棒性与综合优度）

TOPSIS 综合评分给出最优模型为 **WNMF**（贴近度 0.791），排名依次为 **WNMF** > **KNN** (0.735) > 迭代 **SVD** (0.518) > **SoftImpute** (0.496) > 均值填补 (0.419)。Top1 与 Top2 的区分度为 5.5%（大于 1%，结论有区分度），权重在 $\pm 10\%$ 范围扰动下 Top1 稳定率 100%（结论稳健）。综合来看，中高缺失率的 **EHR** 补全首选 **WNMF**（非负可解释、全局低秩、精度稳健），而低缺失率的小矩阵场景下 **KNN** 亦是高效之选。这一结论与任务一小矩阵对比中“**KNN** 在低缺失小矩阵最优”的先验认识相互印证。

九、模型检验与灵敏度分析

为检验模型结论对关键参数与假设的稳健性，本文对五类参数做了系统的灵敏度分析，结果汇总于表 8。

表 8 五类关键参数的灵敏度分析结果

参数	扰动范围	结论
NMF 分解秩 r	$\{2, 3, 5, 8, 10, 15\}$	验证 RSE 在 $r = 2$ 取最小 (0.653), $r \geq 5$ 急升, $r^* = 2$ 稳健
列筛选阈值	$\{0.5, 0.7, 0.9, 0.95\}$	0.5 过低丢信息 (RSE 2.87), 0.7 ~ 0.95 稳健 (0.59 ~ 0.65)
缺失率 ρ	10% ~ 50%	各算法 RSE 单调上升, WNMF 增幅最小、最鲁棒
TOPSIS 权重 w_{RSE}	0.3 ~ 0.7	Top1 在 0.3 ~ 0.6 恒为 WNMF, 仅 0.7 时偏向 KNN
KNN 近邻数 K	$\{3, 5, 12, 15\}$	存在最优 K , 目标随 K 增大平缓下降 (0.475 $\rightarrow$ 0.428)

分解秩 r 的稳健性。留出验证 RSE 在 $r = 2$ 处显著最优, $r \geq 5$ 后迅速恶化, 说明 $r^* = 2$ 的选择对随机起点不敏感, 反映了 EHR 数据真实的强低秩本质。

列筛选阈值的稳健性。阈值取 0.5 时仅保留 10 个特征, 过度丢弃信息导致 RSE 高达 2.87; 阈值在 0.7 ~ 0.95 之间时 RSE 稳定在 0.59 ~ 0.65, 前两主成分累计贡献率稳定在 0.42 ~ 0.46, 说明 0.9 的阈值选择处于稳健区间。

缺失率的单调性。五种算法的 RSE 均随缺失率单调上升, 无一出现“缺失升高而误差反降”的异常, 确认掩码实验与评价实现正确; WNMF 的增幅最小, 再次验证其在高缺失率下的鲁棒性。

TOPSIS 权重的稳健性。将精度权重 w_{RSE} 在 0.3 ~ 0.7 间扰动, Top1 在 0.3 ~ 0.6 区间恒为 WNMF, 仅当权重高达 0.7 (极端偏重精度) 时才转为 KNN。由于本文取 0.5 处于稳健区间内, 最优模型结论可靠。

全局结果验证。本文实现了自动化的全局结果验证脚本, 对所有输出 JSON 做一致性检查, 结果全部通过: 无 NaN/Inf 残留; 任务二各主成分贡献率之和为 1、 $r^* = 2$; 任务三主角均落在 $[0, 90^\circ]$ 、累计贡献增益为正; 任务四 RSE 单调、TOPSIS 权重和为 1、贴近度落在 $[0, 1]$ 、结论有区分度。模型在数值与约束两个层面均通过检验。

十、模型评价与推广

10.1 模型优点

方法契合数据本质。本文针对 EHR 数据的双峰稀疏与强低秩特性, 构建加权掩码 NMF 模型仅在观测位置拟合, 避免了普通 NMF 将缺失当零的系统性偏差; 留出交叉验证选得 $r^* = 2$, 与 Hct-Hgb (相关 0.96)、血压三联等高相关结构在数学上自治。

结果非负可解释。NMF 的非负约束使补全值与生命体征的物理意义一致, 因子可解释为隐含生理模式; 补全后所有值经裁剪落入观测值域, 满足临床合理性约束。

结论经多重验证。核心结论既有数值等价验证（PCA 协方差分解与 SVD 差异 3.1×10^{-15} ），又有合理性检验（RSE 单调性、累计贡献增益为正），还有灵敏度与权重扰动的稳健性检验（Top1 稳定率 100%），并与任务一的小矩阵先验结论相互印证，证据链完整。

评价体系科学。任务四采用同组掩码、多次重复、TOPSIS 多准则决策，兼顾精度、稳定性与效率三个维度，避免了单一指标的片面性。

10.2 模型缺点与局限

其一，分解秩较小（ $r^* = 2$ ）虽契合本数据集，但若推广到低秩性较弱的 EHR 数据集，需重新交叉验证选秩。其二，WNMF 假设缺失机制为 MAR，对于非随机缺失（如某指标因病情危重才被测量）可能产生偏差。其三，受计算资源限制，任务四的基准评价采用 200 行子样本，样本规模的进一步扩大有望使结论更稳定。

10.3 模型推广

本文构建的“掩码低秩补全 + 多准则算法择优”框架不局限于脓毒症数据，可推广至其他高缺失的临床与生物学数据场景，如肿瘤无创早筛指标补全、慢病随访数据修复等。在方法层面，加权掩码 NMF 可进一步引入正则化（如图正则、时序平滑）以利用样本间相似性^[1]与时序连续性；近年基于注意力机制的深度补全方法^[11]在表格型 EHR 上展现出潜力，可与本文的低秩方法形成互补——低秩方法计算高效、可解释性强，适合作为基线和高缺失场景的稳健选择，深度方法则适合在数据充足时进一步提升精度。在评价层面，TOPSIS 多准则框架可灵活纳入更多维度（如可解释性、对非随机缺失的鲁棒性），为不同应用场景下的补全算法选型提供通用决策工具。

参考文献

- [1] Jazayeri A, Liang O S, Yang C C. Interpatient similarity-based imputation of missing data in electronic health records[C]//2019 IEEE International Conference on Healthcare Informatics (ICHI). IEEE, 2019: 1-3.
- [2] Sterne J A C, White I R, Carlin J B, et al. Multiple imputation for missing data in epidemiological and clinical research: potential and pitfalls[J]. BMJ, 2009, 338: b2393.
- [3] Austin P C, White I R, Lee D S, et al. Missing data in clinical research: a tutorial on multiple imputation[J]. Canadian Journal of Cardiology, 2021, 37(9): 1322-1331.

- [4] Mazumder R, Saldana D, Weng H. Matrix completion with nonconvex regularization: spectral operators and scalable algorithms[J]. Statistics and Computing, 2020, 30(4): 1113-1138.
- [5] Lee D D, Seung H S. Learning the parts of objects by non-negative matrix factorization[J]. Nature, 1999, 401(6755): 788-791.
- [6] Reyna M A, Josef C, Seyedi S, et al. Early prediction of sepsis from clinical data: the PhysioNet/Computing in Cardiology Challenge 2019[C]//2019 Computing in Cardiology (CinC). 2019, 45: 1-4.
- [7] Mazumder R, Hastie T, Tibshirani R. Spectral regularization algorithms for learning large incomplete matrices[J]. Journal of Machine Learning Research, 2010, 11: 2287-2322.
- [8] Troyanskaya O, Cantor M, Sherlock G, et al. Missing value estimation methods for DNA microarrays[J]. Bioinformatics, 2001, 17(6): 520-525.
- [9] Zhang X, Xu Z. Extension of TOPSIS to multiple criteria decision making with Pythagorean fuzzy sets[J]. International Journal of Intelligent Systems, 2014, 29(12): 1061-1078.
- [10] Moreira J, Silva B, Faria H, et al. Systematic review on the applicability of principal component analysis for the study of movement in the older adult population[J]. Sensors, 2023, 23(1): 205.
- [11] Kowsar I, Rabbani S B, Samad M D. Attention-based imputation of missing values in electronic health records tabular data[C]//2024 IEEE 12th International Conference on Healthcare Informatics (ICHI). IEEE, 2024: 177-182.

附录 A 核心代码

本附录给出本文四个任务的核心 Python 实现代码。完整工程包含数据载入 (`data_loader.py`)、工具函数与各算法实现 (`utils.py`)、四个任务主程序 (`problem1_4.py`)、灵敏度分析 (`sensitivity_analysis.py`) 与全局验证 (`validate.py`)。运行方式: `cd code && python main.py`。

1.1 工具函数与核心算法 (`utils.py`)

```
1 # -*- coding: utf-8 -*-
2 # __mh_autobootstrap_syspath__
3 import os as _mh_os, sys as _mh_sys
```

```

4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """公共工具：归一化、补全算法、评价指标、约束验证。
9
10 物理参数 / 方法来源：MODELING_REPORT.md（第 3~8 节）。
11 所有补全算法仅利用观测项统计量，禁止用缺失位污染统计量。
12  """
13  # 自举模块路径（让 sibling import 不依赖调用方式）
14  import os, sys
15  _HERE = os.path.dirname(os.path.abspath(__file__))
16  if _HERE not in sys.path:
17      sys.path.insert(0, _HERE)
18
19  import numpy as np
20
21  EPS = 1e-9
22  RANDOM_SEED = 42
23
24
25  # =====
26  # 归一化（统计量仅用观测项；缺失=np.nan）
27  # =====
28  def minmax_normalize(A):
29      """Min-Max 归一化到 [0,1]（用于 NMF，保非负）。返回 (A_norm, mins, maxs)。"""
30      mins = np.nanmin(A, axis=0)
31      maxs = np.nanmax(A, axis=0)
32      rng = maxs - mins
33      rng[rng == 0] = 1.0 # 常数列防零除
34      A_norm = (A - mins) / rng
35      return A_norm, mins, maxs
36
37
38  def minmax_inverse(A_norm, mins, maxs):
39      """逆 Min-Max 还原量纲。"""
40      rng = maxs - mins
41      rng[rng == 0] = 1.0
42      return A_norm * rng + mins
43
44
45  def zscore_normalize(A):
46      """Z-score 标准化（用于 PCA），统计量仅用观测项。返回 (Z, mu, sigma)。"""
47      mu = np.nanmean(A, axis=0)
48      sigma = np.nanstd(A, axis=0, ddof=1)
49      sigma[sigma == 0] = 1.0
50      Z = (A - mu) / sigma

```

```

51     return Z, mu, sigma
52
53
54 # =====
55 # 补全算法（任务1 四法 + 任务4 横评）
56 # =====
57 def impute_mean(A):
58     """均值填补（基线）。仅用观测项均值。"""
59     out = A.copy()
60     col_mean = np.nanmean(A, axis=0)
61     col_mean = np.where(np.isnan(col_mean), 0.0, col_mean)
62     idx = np.where(np.isnan(out))
63     out[idx] = np.take(col_mean, idx[1])
64     return out
65
66
67 def impute_knn(A, K=None):
68     """KNN 距离加权填补。在观测子空间用欧氏距离找 K 近邻。"""
69     m, n = A.shape
70     if K is None:
71         K = max(1, int(np.floor(np.sqrt(m))))
72     out = A.copy()
73     obs = ~np.isnan(A)
74     col_mean = np.nanmean(A, axis=0)
75     col_mean = np.where(np.isnan(col_mean), 0.0, col_mean)
76     for i in range(m):
77         miss_j = np.where(~obs[i])[0]
78         if len(miss_j) == 0:
79             continue
80         dists = []
81         for k in range(m):
82             if k == i:
83                 continue
84             common = obs[i] & obs[k]
85             if common.sum() == 0:
86                 continue
87             d = np.sqrt(np.mean((A[i, common] - A[k, common])**2))
88             dists.append((d, k))
89         dists.sort(key=lambda t: t[0])
90         for j in miss_j:
91             num = den = 0.0
92             cnt = 0
93             for d, k in dists:
94                 if obs[k, j]:
95                     w = 1.0 / (d + EPS)
96                     num += w * A[k, j]
97                     den += w

```

```

98         cnt += 1
99         if cnt >= K:
100             break
101         out[i, j] = num / den if den > 0 else col_mean[j]
102     return out
103
104
105 def impute_iterative_svd(A, rank=None, tol=1e-4, max_iter=200, tau=None):
106     """迭代 SVD 低秩补全 / SoftImpute (奇异值软阈值)。"""
107     m, n = A.shape
108     obs = ~np.isnan(A)
109     col_mean = np.nanmean(A, axis=0)
110     col_mean = np.where(np.isnan(col_mean), 0.0, col_mean)
111     cur = A.copy()
112     idx_miss = np.where(~obs)
113     cur[idx_miss] = np.take(col_mean, idx_miss[1])
114     if rank is None:
115         rank = max(1, min(m, n) // 2)
116     for _ in range(max_iter):
117         U, s, Vt = np.linalg.svd(cur, full_matrices=False)
118         if tau is not None:
119             s = np.maximum(s - tau, 0.0) # 软阈值
120         else:
121             s[rank:] = 0.0 # 硬截断到 rank
122         recon = (U * s) @ Vt
123         new = cur.copy()
124         new[idx_miss] = recon[idx_miss] # 仅回填缺失位, 观测位保留
125         denom = np.linalg.norm(cur) + EPS
126         if np.linalg.norm(new - cur) / denom < tol:
127             cur = new
128             break
129     cur = new
130     return cur
131
132
133 def impute_linear_interp(A):
134     """沿行方向 (时间) 线性插值, 端点最近邻外延; 整列全缺则用列均值 (此处 0)。"""
135     m, n = A.shape
136     out = A.copy()
137     col_mean = np.nanmean(A, axis=0)
138     col_mean = np.where(np.isnan(col_mean), 0.0, col_mean)
139     rows = np.arange(m)
140     for j in range(n):
141         col = out[:, j]
142         obs = ~np.isnan(col)
143         if obs.sum() == 0:
144             out[:, j] = col_mean[j]

```

```

145     elif obs.sum() == 1:
146         out[:, j] = col[obs][0]
147     else:
148         out[:, j] = np.interp(rows, rows[obs], col[obs])
149     return out
150
151
152 def softimpute(A, tau=None, tol=1e-4, max_iter=200):
153     """SoftImpute (核范数软阈值), tau 默认取首奇异值的一定比例。"""
154     if tau is None:
155         tmp = A.copy()
156         cm = np.nanmean(A, axis=0)
157         cm = np.where(np.isnan(cm), 0.0, cm)
158         idx = np.where(np.isnan(tmp))
159         tmp[idx] = np.take(cm, idx[1])
160         s0 = np.linalg.svd(tmp, compute_uv=False)
161         tau = 0.1 * s0[0]
162     return impute_iterative_svd(A, tol=tol, max_iter=max_iter, tau=tau)
163
164
165 # =====
166 # 加权掩码 NMF (WNMF, Lee-Seung 乘性更新 + 掩码 + 多起点)
167 # =====
168 def wnmf(A, Omega, r, max_iter=500, tol=1e-5, n_init=5, seed=RANDOM_SEED):
169     """加权掩码 NMF。仅在观测位 (Omega==1) 拟合 A=WH, W,H>=0。
170
171     返回 (W, H, A_hat_norm, loss_history_best)。
172     """
173     m, n = A.shape
174     A_obs = np.where(Omega > 0, A, 0.0)
175     best = None
176     best_loss = np.inf
177     best_hist = None
178     rng = np.random.default_rng(seed)
179     for start in range(n_init):
180         W = rng.random((m, r)) + EPS
181         H = rng.random((r, n)) + EPS
182         hist = []
183         prev_f = None
184         for t in range(max_iter):
185             WH = W @ H
186             # H 更新
187             numH = W.T @ A_obs
188             denH = W.T @ (Omega * WH) + EPS
189             H = H * (numH / denH)
190             WH = W @ H
191             # W 更新

```

```

192     numW = A_obs @ H.T
193     denW = (Omega * WH) @ H.T + EPS
194     W = W * (numW / denW)
195     WH = W @ H
196     f = np.sum((Omega * (A - WH)) ** 2)
197     hist.append(float(f))
198     if prev_f is not None and prev_f > 0 and abs(prev_f - f) / prev_f < tol:
199         break
200     prev_f = f
201     final_f = hist[-1]
202     if final_f < best_loss:
203         best_loss = final_f
204         best = (W.copy(), H.copy())
205         best_hist = hist
206 W, H = best
207 A_hat = Omega * A + (1 - Omega) * (W @ H) # 观测位保留, 缺失位填重构
208 return W, H, A_hat, best_hist
209
210
211 def wnmf_select_rank(A, Omega, ranks=(2, 3, 5, 8, 10, 15), val_frac=0.1,
212                     seed=RANDOM_SEED, **kw):
213     """留出交叉验证选秩: 从已观测元素留出 val_frac 作验证集, 按验证 RSE 选 r*."""
214     rng = np.random.default_rng(seed)
215     obs_idx = np.argwhere(Omega > 0)
216     n_val = max(1, int(len(obs_idx) * val_frac))
217     perm = rng.permutation(len(obs_idx))
218     val_sel = obs_idx[perm[:n_val]]
219     Omega_train = Omega.copy()
220     for (i, j) in val_sel:
221         Omega_train[i, j] = 0
222     a_val = np.array([A[i, j] for i, j in val_sel])
223     a_bar = a_val.mean()
224     results = {}
225     for r in ranks:
226         _, _, A_hat, _ = wnmf(A, Omega_train, r, seed=seed, **kw)
227         pred = np.array([A_hat[i, j] for i, j in val_sel])
228         denom = np.sum((a_val - a_bar) ** 2) + EPS
229         rse = float(np.sum((pred - a_val) ** 2) / denom)
230         results[r] = rse
231     r_star = min(results, key=results.get)
232     return r_star, results
233
234
235 # =====
236 # 评价指标
237 # =====
238 def rse(pred, true):

```

```

239     """相对平方误差。"""
240     pred, true = np.asarray(pred, float), np.asarray(true, float)
241     bar = true.mean()
242     denom = np.sum((true - bar) ** 2) + EPS
243     return float(np.sum((pred - true) ** 2) / denom)
244
245
246 def rmse(pred, true):
247     pred, true = np.asarray(pred, float), np.asarray(true, float)
248     return float(np.sqrt(np.mean((pred - true) ** 2)))
249
250
251 # =====
252 # PCA (协方差特征分解)
253 # =====
254 def pca_cov(Z):
255     """对 Z-score 矩阵做协方差特征分解。返回 (eigvals, eigvecs, var_ratio, scores)。
256
257     特征值降序; eigvecs 列为特征向量 (单位化); scores = Z @ eigvecs。
258     """
259     Z = np.nan_to_num(Z, nan=0.0)
260     m = Z.shape[0]
261     Sigma = (Z.T @ Z) / (m - 1)
262     Sigma = (Sigma + Sigma.T) / 2 # 强制对称
263     eigvals, eigvecs = np.linalg.eigh(Sigma)
264     order = np.argsort(eigvals)[::-1]
265     eigvals = eigvals[order]
266     eigvecs = eigvecs[:, order]
267     eigvals = np.clip(eigvals, 0.0, None) # 半正定, 去除数值负值
268     var_ratio = eigvals / (eigvals.sum() + EPS)
269     scores = Z @ eigvecs
270     return eigvals, eigvecs, var_ratio, scores
271
272
273 # =====
274 # 约束验证 (每个子问题末尾调用, 来源 MODELING_REPORT 第8节)
275 # =====
276 def validate_constraints(results, constraints):
277     """results: {key: value}; constraints: {key: (lo, hi, desc)}. 返回 True/False. """
278     violations = []
279     for key, (lo, hi, desc) in constraints.items():
280         val = results.get(key)
281         if val is None:
282             continue
283         vals = np.atleast_1d(np.asarray(val, float))
284         for v in vals.ravel():
285             if np.isnan(v):

```

```

286         violations.append(f" {desc}: 出现 NaN")
287     elif v < lo - 1e-6 or v > hi + 1e-6:
288         violations.append(f" {desc}: {v} 超出范围 [{lo}, {hi}]")
289     if violations:
290         print("\n" + "=" * 60)
291         print(" 约束验证失败")
292         for v in violations:
293             print(" " + v)
294         print("=" * 60)
295         return False
296     print(" 约束验证通过")
297     return True
298
299
300 def to_jsonable(obj):
301     """把 numpy 类型递归转成 JSON 可序列化类型。"""
302     if isinstance(obj, dict):
303         return {str(k): to_jsonable(v) for k, v in obj.items()}
304     if isinstance(obj, (list, tuple)):
305         return [to_jsonable(v) for v in obj]
306     if isinstance(obj, np.ndarray):
307         return to_jsonable(obj.tolist())
308     if isinstance(obj, (np.integer,)):
309         return int(obj)
310     if isinstance(obj, (np.floating,)):
311         return float(obj)
312     if isinstance(obj, (np.bool_,)):
313         return bool(obj)
314     return obj

```

1.2 任务一：数据预处理与补全（problem1.py）

```

1  # -*- coding: utf-8 -*-
2  # __mh_autobootstrap_syspath__
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """任务1：数据预处理 — 缺失率统计、归一化、给定矩阵 4 法补全对比。
9
10 来源：MODELING_REPORT 第3节。
11  """
12  import os, sys
13  _HERE = os.path.dirname(os.path.abspath(__file__))

```

```

14 if _HERE not in sys.path:
15     sys.path.insert(0, _HERE)
16
17 import json
18 import numpy as np
19
20 import data_loader as dl
21 import utils as u
22
23 OUT = os.path.join(_HERE, '..', 'figures')
24 os.makedirs(OUT, exist_ok=True)
25
26
27 def run():
28     print('[Task1] 开始 ...')
29     z = dl._open_zip()
30
31     # ---- 模型A: p000001 按列缺失率统计 ----
32     df = dl.get_p000001(z)
33     miss_all = df.iloc[:, :40].isna().mean() # 前40生命特征
34     first36 = list(df.columns[:36])
35     sel11 = [(c, float(miss_all[c])) for c in first36 if miss_all[c] < dl.MISS_COL_THRESH]
36     print(f'[Task1] p000001 前36特征中缺失率<90% 的有 {len(sel11)} 个')
37
38     # 全40特征缺失率 (供 fig_missing_bar)
39     miss40 = {c: float(miss_all[c]) for c in df.columns[:40]}
40
41     # ---- 构造 50 份 36x41 标准矩阵 (供后续任务复用, 这里输出规模信息) ----
42     mats, used, _plabels = dl.collect_standard_matrices(n_samples=50, z=z)
43     print(f'[Task1] 筛选标准样本 {len(mats)} 份, 形如 {mats[0].shape}')
44
45     # ---- 模型B: 归一化演示 (取首个样本前36特征) ----
46     sample = mats[0][:, :40]
47     A_mm, mins, maxs = u.minmax_normalize(sample)
48     Z, mu, sigma = u.zscore_normalize(sample)
49     # 归一化后范围验证 (仅观测项)
50     mm_obs = A_mm[~np.isnan(A_mm)]
51     print(f'[Task1] Min-Max 后观测项范围: [{mm_obs.min():.3f}, {mm_obs.max():.3f}']')
52
53     # ---- 模型C: 给定矩阵 4 法补全对比 ----
54     M_nan, M_true, mask, mcols = dl.get_given_matrix()
55     miss_pos = np.argwhere(mask == 0)
56     true_vals = np.array([M_true[i, j] for i, j in miss_pos])
57
58     methods = {}
59
60     def eval_method(name, M_filled):

```

```

61     pred = np.array([M_filled[i, j] for i, j in miss_pos])
62     methods[name] = {
63         'rmse': u.rmse(pred, true_vals),
64         'rse': u.rse(pred, true_vals),
65         'mae': float(np.mean(np.abs(pred - true_vals))),
66         'pred': pred.tolist(),
67     }
68
69     eval_method('均值填补', u.impute_mean(M_nan))
70     eval_method('KNN填补', u.impute_knn(M_nan, K=3))
71     eval_method('迭代SVD', u.impute_iterative_svd(M_nan, rank=2))
72     eval_method('线性插值', u.impute_linear_interp(M_nan))
73
74     print('[Task1] 给定矩阵 4 法补全 RMSE:')
75     for k, v in methods.items():
76         print(f'    {k}: RMSE={v["rmse"]:.4f}, RSE={v["rse"]:.4f}, MAE={v["mae"]:.4f}')
77
78     z.close()
79
80     # ---- 约束验证 ----
81     # 补全值应落在该演示矩阵观测值域内（生理合理）
82     col_min = np.nanmin(M_true, axis=0)
83     col_max = np.nanmax(M_true, axis=0)
84     constraints = {}
85     check_vals = {}
86     for name, v in methods.items():
87         arr = np.array(v['pred'])
88         # 用每个缺失位对应列的界
89         lo = np.array([col_min[j] for _, j in miss_pos])
90         hi = np.array([col_max[j] for _, j in miss_pos])
91         # 越界量（仅诊断，最终结果裁剪在合理域）
92         over = np.maximum(arr - hi, 0) + np.maximum(lo - arr, 0)
93         check_vals[f'{name}_max_over'] = float(over.max())
94     # RMSE/RSE/MAE 非负
95     for name, v in methods.items():
96         for met in ['rmse', 'rse', 'mae']:
97             check_vals[f'{name}_{met}'] = v[met]
98             constraints[f'{name}_{met}'] = (0.0, 1e9, f'{name}_{met}')
99     passed = u.validate_constraints(check_vals, constraints)
100     if not passed:
101         raise ValueError('[Task1] 约束验证失败')
102
103     result = {
104         'task': 1,
105         'p000001_shape': list(df.shape),
106         'miss_threshold': d1.MISS_COL_THRESH,
107         'p000001_first36_lt90_count': len(sel11),

```

```

108     'p000001_key11_features': [{'feature': c, 'miss_rate': r} for c, r in sel11],
109     'miss_rate_all40': miss40,
110     'n_standard_matrices': len(mats),
111     'standard_matrix_shape': list(mats[0].shape),
112     'sample_files': used[:10],
113     'normalization': {
114         'minmax_obs_range': [float(mm_obs.min()), float(mm_obs.max())],
115         'zscore_demo_mu': u.to_jsonable(np.nan_to_num(mu[:11], nan=0.0)),
116     },
117     'given_matrix': {
118         'columns': mcols,
119         'shape': list(M_true.shape),
120         'n_missing': int((mask == 0).sum()),
121         'true_values_at_missing': true_vals.tolist(),
122         'methods': {k: {kk: vv for kk, vv in v.items()} for k, v in methods.items()},
123     },
124     'method_characteristics': {
125         '均值填补': '不利用相关结构, 0(mn), 低估方差, 仅基线',
126         'KNN填补': '利用局部样本相似, 对中低缺失率好, 高缺失率邻域不可靠',
127         '迭代SVD': '利用全局低秩结构, 对高缺失率鲁棒, 计算开销中等',
128         '线性插值': '利用时序连续性, 适合连续监护列, 不适合偶测化验列',
129     },
130 }
131
132 path = os.path.join(OUT, 'problem_1_results.json')
133 with open(path, 'w', encoding='utf-8') as f:
134     json.dump(u.to_jsonable(result), f, ensure_ascii=False, indent=2)
135     print(f'[Task1] 结果已保存 -> {path}')
136     return result
137
138
139 if __name__ == '__main__':
140     run()

```

1.3 任务二：加权掩码 NMF 补全与 PCA (problem2.py)

```

1  # -*- coding: utf-8 -*-
2  # __mh_autobootstrap_syspath__
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """任务2: 加权掩码 NMF 全量补全 + PCA 主成分分析。
9

```

```

10 来源: MODELING_REPORT 第4节。
11 流程: 构造观测矩阵 A(列筛选<90%) -> Min-Max -> 留出CV选秩 r* -> WNMF全量补全
12      -> 逆归一化 + 裁剪到观测值域 -> Z-score -> 协方差特征分解 PCA。
13  """
14  import os, sys, warnings
15  _HERE = os.path.dirname(os.path.abspath(__file__))
16  if _HERE not in sys.path:
17      sys.path.insert(0, _HERE)
18  warnings.filterwarnings('ignore', category=RuntimeWarning)
19
20  import json
21  import numpy as np
22
23  import data_loader as dl
24  import utils as u
25
26  OUT = os.path.join(_HERE, '..', 'figures')
27  os.makedirs(OUT, exist_ok=True)
28
29
30  def build_completed_dataset(n_samples=50, seed=dl.RANDOM_SEED):
31      """构造观测矩阵 A 并用 WNMF 全量补全。返回补全所需的全部中间量（供任务3/4 复用）。"""
32      z = dl._open_zip()
33      mats, used, plabels = dl.collect_standard_matrices(n_samples=n_samples, z=z, seed=seed)
34      z.close()
35      A_raw, Omega, feat_names, label_vec = dl.build_observation_matrix(mats, plabels)
36      print(f'[Task2] 观测矩阵 A: {A_raw.shape}, 有效特征 {len(feat_names)} 个')
37      print(f'[Task2] 整体观测率: {Omega.mean():.3f}')
38
39      # Min-Max 归一化（保非负，统计量仅用观测项）
40      A_norm, mins, maxs = u.minmax_normalize(A_raw)
41      A_norm = np.nan_to_num(A_norm, nan=0.0) # 缺失位填0仅占位（被Omega屏蔽，不参与拟合）
42
43      # 留出 CV 选秩
44      r_star, rse_by_r = u.wnmf_select_rank(
45          A_norm, Omega, ranks=(2, 3, 5, 8, 10, 15), val_frac=0.1, seed=seed)
46      print(f'[Task2] 选秩 r* = {r_star}, 验证RSE={rse_by_r}')
47
48      # 用 r* 做全量 WNMF 补全
49      W, H, A_hat_norm, loss_hist = u.wnmf(A_norm, Omega, r_star, seed=seed)
50
51      # 逆 Min-Max 还原量纲
52      A_hat = u.minmax_inverse(A_hat_norm, mins, maxs)
53      # 观测位用原值
54      A_hat = np.where(Omega > 0, A_raw, A_hat)
55
56      # 裁剪到每特征观测值域 [min_j, max_j]（假设5: 生理合理）

```

```

57 clip_lo = np.nanmin(A_raw, axis=0)
58 clip_hi = np.nanmax(A_raw, axis=0)
59 A_hat_clipped = np.clip(A_hat, clip_lo, clip_hi)
60 n_clipped = int(np.sum(np.abs(A_hat_clipped - A_hat) > 1e-9))
61 A_hat = A_hat_clipped
62
63 return {
64     'A_raw': A_raw, 'Omega': Omega, 'feat_names': feat_names,
65     'label_vec': label_vec, 'A_hat': A_hat, 'W': W, 'H': H,
66     'r_star': r_star, 'rse_by_r': rse_by_r, 'loss_hist': loss_hist,
67     'mins': mins, 'maxs': maxs, 'n_clipped': n_clipped, 'used': used,
68     'A_norm': A_norm, 'A_hat_norm': A_hat_norm,
69 }
70
71
72 def run():
73     print('[Task2] 开始 ...')
74     D = build_completed_dataset()
75     A_hat = D['A_hat']
76     feat_names = D['feat_names']
77
78     # 观测位重构相对误差（模型检验）：用 WH 重构与观测真值比较（非合并后的 A_hat）
79     Omega = D['Omega']
80     A_norm = D['A_norm']
81     WH = D['W'] @ D['H']
82     recon_err = (np.linalg.norm(Omega * (A_norm - WH)) /
83                  (np.linalg.norm(Omega * A_norm) + u.EPS))
84     print(f'[Task2] 观测位重构相对误差 ||Ω(A-WH)||/||ΩA|| = {recon_err:.4f}')
85
86     # 无 NaN 检查
87     assert not np.isnan(A_hat).any(), '补全后仍有 NaN'
88
89     # ---- PCA（对补全后完整数据，Z-score） ----
90     Z, mu, sigma = u.zscore_normalize(A_hat)
91     eigvals, eigvecs, var_ratio, scores = u.pca_cov(Z)
92     cum = np.cumsum(var_ratio)
93     print(f'[Task2] PC1贡献={var_ratio[0]:.4f}, PC2贡献={var_ratio[1]:.4f},
94           累计前2={cum[1]:.4f}')
95
96     # 载荷 L_jk = sqrt(lambda_k) * v_jk
97     loadings = (eigvecs * np.sqrt(eigvals)[None, :])
98
99     # PCA 一致性检验：协方差特征分解 vs Z 的 SVD
100     U_svd, s_svd, _ = np.linalg.svd(np.nan_to_num(Z), full_matrices=False)
101     lam_svd = s_svd ** 2 / (Z.shape[0] - 1)
102     consistency = float(np.max(np.abs(np.sort(eigvals)[::-1][:5] - lam_svd[:5])))
103     print(f'[Task2] PCA一致性(cov vs svd) 最大差异={consistency:.2e}')

```

```

103
104 # ---- 约束验证 ----
105 check = {
106     'var_ratio_sum': float(var_ratio.sum()),
107     'eigvals_min': float(eigvals.min()),
108     'var_ratio_max': float(var_ratio.max()),
109     'recon_err': recon_err,
110     'W_min': float(D['W'].min()),
111     'H_min': float(D['H'].min()),
112     'eigvec_norm': float(np.linalg.norm(eigvecs[:, 0])),
113 }
114 constraints = {
115     'var_ratio_sum': (1 - 1e-6, 1 + 1e-6, '方差贡献率之和=1'),
116     'eigvals_min': (0.0, 1e12, '特征值非负'),
117     'var_ratio_max': (0.0, 1.0, '方差贡献率<=1'),
118     'recon_err': (0.0, 1.0, '观测位重构相对误差'),
119     'W_min': (0.0, 1e12, 'W非负'),
120     'H_min': (0.0, 1e12, 'H非负'),
121     'eigvec_norm': (1 - 1e-6, 1 + 1e-6, '特征向量单位化'),
122 }
123 # 补全值生理硬界
124 for fname, (lo, hi) in dl.PHYS_BOUNDS.items():
125     if fname in feat_names:
126         j = feat_names.index(fname)
127         col = A_hat[:, j]
128         check[f'{fname}_min'] = float(col.min())
129         check[f'{fname}_max'] = float(col.max())
130         constraints[f'{fname}_min'] = (lo, hi, f'{fname}下界')
131         constraints[f'{fname}_max'] = (lo, hi, f'{fname}上界')
132 passed = u.validate_constraints(check, constraints)
133 if not passed:
134     raise ValueError('[Task2] 约束验证失败')
135
136 # 保存补全后数据集（供任务3/4 复用），用 npz
137 np.savez(os.path.join(OUT, '_task2_completed.npz'),
138          A_hat=A_hat, A_raw=D['A_raw'], Omega=Omega,
139          label_vec=D['label_vec'], feat_names=np.array(feat_names, dtype=object))
140
141 n = len(feat_names)
142 result = {
143     'task': 2,
144     'A_shape': list(D['A_raw'].shape),
145     'n_features': n,
146     'feat_names': feat_names,
147     'overall_obs_rate': float(Omega.mean()),
148     'r_star': D['r_star'],
149     'rse_by_rank': {str(k): v for k, v in D['rse_by_r'].items()},

```

```

150     'wnmf_loss_history': [float(x) for x in D['loss_hist']],
151     'recon_error_observed': recon_err,
152     'n_values_clipped': D['n_clipped'],
153     'pca': {
154         'eigenvalues': u.to_jsonable(eigvals),
155         'variance_ratio': u.to_jsonable(var_ratio),
156         'cumulative_ratio': u.to_jsonable(cum),
157         'pc1_loadings': {feat_names[j]: float(loadings[j, 0]) for j in range(n)},
158         'pc2_loadings': {feat_names[j]: float(loadings[j, 1]) for j in range(n)},
159         'pc1_eigvec': u.to_jsonable(eigvecs[:, 0]),
160         'pc2_eigvec': u.to_jsonable(eigvecs[:, 1]),
161         'scores_pc1': u.to_jsonable(scores[:, 0]),
162         'scores_pc2': u.to_jsonable(scores[:, 1]),
163         'consistency_cov_vs_svd': consistency,
164     },
165     'label_vec': u.to_jsonable(D['label_vec']),
166 }
167 path = os.path.join(OUT, 'problem_2_results.json')
168 with open(path, 'w', encoding='utf-8') as f:
169     json.dump(u.to_jsonable(result), f, ensure_ascii=False, indent=2)
170 print(f'[Task2] 结果已保存 -> {path}')
171 return result
172
173
174 if __name__ == '__main__':
175     run()

```

1.4 任务三：临床规律挖掘与补全增益评价（problem3.py）

```

1  # -*- coding: utf-8 -*-
2  # __mh_autobootstrap_syspath__
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """任务3：基于 PCA 结果的临床规律挖掘 + 补全前后 PCA 对比（量化补全增益）。
9
10 来源：MODELING_REPORT 第5节。
11 1. 载荷阈值筛选关键预警指标 + SepsisLabel 关联（Mann-Whitney U / 点二列相关）。
12 2. 补全前（成对删除协方差）vs 补全后 PCA 对比：贡献率差、主角、Grassmann距离。
13 """
14 import os, sys, warnings
15 _HERE = os.path.dirname(os.path.abspath(__file__))
16 if _HERE not in sys.path:

```

```

17     sys.path.insert(0, _HERE)
18 warnings.filterwarnings('ignore', category=RuntimeWarning)
19
20 import json
21 import numpy as np
22 from scipy import stats
23 from scipy.linalg import subspace_angles
24
25 import utils as u
26
27 OUT = os.path.join(_HERE, '..', 'figures')
28 TAU_L = 0.5 # 载荷阈值
29
30
31 def pca_pairwise_before(A_raw):
32     """补全前：用成对删除(pairwise)估计协方差，做 PCA（处理缺失）。"""
33     n = A_raw.shape[1]
34     # 仅观测项的均值/标准差标准化
35     mu = np.nanmean(A_raw, axis=0)
36     sigma = np.nanstd(A_raw, axis=0, ddof=1)
37     sigma[sigma == 0] = 1.0
38     Zb = (A_raw - mu) / sigma
39     # 成对协方差
40     Sigma = np.zeros((n, n))
41     for i in range(n):
42         for j in range(n):
43             mask = ~np.isnan(Zb[:, i]) & ~np.isnan(Zb[:, j])
44             if mask.sum() > 1:
45                 Sigma[i, j] = np.mean(Zb[mask, i] * Zb[mask, j])
46     Sigma = (Sigma + Sigma.T) / 2
47     eigvals, eigvecs = np.linalg.eigh(Sigma)
48     order = np.argsort(eigvals)[::-1]
49     eigvals = np.clip(eigvals[order], 0, None)
50     eigvecs = eigvecs[:, order]
51     var_ratio = eigvals / (eigvals.sum() + u.EPS)
52     return eigvals, eigvecs, var_ratio
53
54
55 def principal_angle(v_before, v_after):
56     """两向量主角 = arccos(|<v_b, v_a>|), 弧度。"""
57     cos = np.abs(np.dot(v_before, v_after) /
58                  (np.linalg.norm(v_before) * np.linalg.norm(v_after) + u.EPS))
59     cos = np.clip(cos, 0, 1)
60     return float(np.arccos(cos))
61
62
63 def run():

```

```

64 print('[Task3] 开始 ...')
65 npz = np.load(os.path.join(OUT, '_task2_completed.npz'), allow_pickle=True)
66 A_hat = npz['A_hat']
67 A_raw = npz['A_raw']
68 label_vec = npz['label_vec']
69 feat_names = list(npz['feat_names'])
70 n = len(feat_names)
71
72 # ---- 补全后 PCA ----
73 Z_after, _, _ = u.zscore_normalize(A_hat)
74 ev_a, evec_a, vr_a, scores_a = u.pca_cov(Z_after)
75 loadings_a = evec_a * np.sqrt(ev_a)[None, :]
76
77 # ---- 关键预警指标筛选 (按 max(|L_jk|, k=1,2) >= TAU_L) ----
78 key_score = np.max(np.abs(loadings_a[:, :2]), axis=1)
79 key_idx = [j for j in range(n) if key_score[j] >= TAU_L]
80 key_vars = [feat_names[j] for j in key_idx]
81 print(f'[Task3] 高载荷候选预警指标({len(key_vars)}个): {key_vars}')
82
83 # ---- SepsisLabel 关联验证 (Mann-Whitney U + 点二列相关) ----
84 y = label_vec.astype(float)
85 valid = ~np.isnan(y)
86 y = y[valid]
87 assoc = {}
88 for j in range(n):
89     col = A_hat[valid, j]
90     g1 = col[y == 1]
91     g0 = col[y == 0]
92     if len(g1) < 5 or len(g0) < 5:
93         continue
94     try:
95         ustat, pval = stats.mannwhitneyu(g1, g0, alternative='two-sided')
96     except ValueError:
97         continue
98     # 点二列相关
99     r_pb, _ = stats.pointbiserialr(y, col)
100     assoc[feat_names[j]] = {
101         'mannwhitney_p': float(pval),
102         'pointbiserial_r': float(r_pb),
103         'mean_sepsis': float(np.mean(g1)),
104         'mean_nonsepsis': float(np.mean(g0)),
105         'high_loading': feat_names[j] in key_vars,
106         'max_abs_loading': float(key_score[j]),
107     }
108 # 最终预警指标: 高载荷 且 与标签显著关联(p<0.05)
109 final_warning = [f for f in key_vars
110                  if f in assoc and assoc[f]['mannwhitney_p'] < 0.05]

```

```

111 print(f'[Task3] 最终生理预警指标(高载荷+显著关联): {final_warning}')
112
113 # ---- 补全前后 PCA 对比 ----
114 ev_b, evec_b, vr_b = pca_pairwise_before(A_raw)
115
116 # 方差贡献率差
117 delta_eta = (vr_a[:5] - vr_b[:5]).tolist()
118 cum2_before = float(np.sum(vr_b[:2]))
119 cum2_after = float(np.sum(vr_a[:2]))
120
121 # 主角 (逐向量 PC1, PC2)
122 theta1 = principal_angle(evec_b[:, 0], evec_a[:, 0])
123 theta2 = principal_angle(evec_b[:, 1], evec_a[:, 1])
124 grassmann = float(np.sqrt(theta1 ** 2 + theta2 ** 2))
125 # 子空间主角 (PC1-PC2 子空间整体, 对平面内旋转鲁棒)
126 sub_ang = subspace_angles(evec_b[:, :2], evec_a[:, :2])
127 sub_ang_deg = sorted(np.degrees(sub_ang).tolist())
128 print(f'[Task3] 累计前2贡献 补全前={cum2_before:.4f} -> 补全后={cum2_after:.4f}')
129 print(f'[Task3] 逐向量主角 1={np.degrees(theta1):.2f}°, 2={np.degrees(theta2):.2f}°')
130 print(f'[Task3] PC1-PC2子空间主角 = {sub_ang_deg} (度)')
131
132 # ---- 约束验证 ----
133 check = {
134     'vr_after_sum': float(vr_a.sum()),
135     'vr_before_sum': float(vr_b.sum()),
136     'theta1_deg': float(np.degrees(theta1)),
137     'theta2_deg': float(np.degrees(theta2)),
138     'cum2_after': cum2_after,
139 }
140 constraints = {
141     'vr_after_sum': (1 - 1e-6, 1 + 1e-6, '补全后贡献率和=1'),
142     'vr_before_sum': (1 - 1e-6, 1 + 1e-6, '补全前贡献率和=1'),
143     'theta1_deg': (0.0, 90.0, '主角 1在[0,90]'),
144     'theta2_deg': (0.0, 90.0, '主角 2在[0,90]'),
145     'cum2_after': (0.0, 1.0, '累计贡献<=1'),
146 }
147 if not u.validate_constraints(check, constraints):
148     raise ValueError('[Task3] 约束验证失败')
149
150 result = {
151     'task': 3,
152     'tau_loading': TAU_L,
153     'key_warning_candidates': key_vars,
154     'final_warning_indicators': final_warning,
155     'loading_screen': {feat_names[j]: float(key_score[j]) for j in range(n)},
156     'sepsis_association': assoc,
157     'before_after_pca': {

```

```

158         'variance_ratio_before': u.to_jsonable(vr_b),
159         'variance_ratio_after': u.to_jsonable(vr_a),
160         'delta_eta_top5': delta_eta,
161         'cum2_before': cum2_before,
162         'cum2_after': cum2_after,
163         'cum2_gain': cum2_after - cum2_before,
164         'theta1_rad': theta1, 'theta2_rad': theta2,
165         'theta1_deg': float(np.degrees(theta1)),
166         'theta2_deg': float(np.degrees(theta2)),
167         'subspace_principal_angles_deg': sub_ang_deg,
168         'grassmann_distance': grassmann,
169     },
170     'pc1_loadings_after': {feat_names[j]: float(loadings_a[j, 0]) for j in range(n)},
171     'pc2_loadings_after': {feat_names[j]: float(loadings_a[j, 1]) for j in range(n)},
172 }
173 path = os.path.join(OUT, 'problem_3_results.json')
174 with open(path, 'w', encoding='utf-8') as f:
175     json.dump(u.to_jsonable(result), f, ensure_ascii=False, indent=2)
176 print(f'[Task3] 结果已保存 -> {path}')
177 return result
178
179
180 if __name__ == '__main__':
181     run()

```

1.5 任务四：多算法多缺失率横评与 TOPSIS (problem4.py)

```

1  # -*- coding: utf-8 -*-
2  # __mh_autobootstrap_syspath__
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """任务4：多算法 × 多缺失率补全精度对比 + TOPSIS 最优模型筛选。
9
10 来源：MODELING_REPORT 第6节。
11 以任务2 完整基准数据 A*(无缺失) 为真值，每档缺失率 {10,20,30,40,50}% 额外随机
12 均匀掩码(MAR)，各算法用同一组掩码补全，仅在人工掩码位与真值比较。每档重复 R>=5 次。
13 评价指标 RSE/RMSE/Tcpu 均为负向，TOPSIS(极差负向无量纲化，权重0.5/0.3/0.2)择优。
14 """
15 import os, sys, time, warnings
16 _HERE = os.path.dirname(os.path.abspath(__file__))
17 if _HERE not in sys.path:
18     sys.path.insert(0, _HERE)

```

```

19 warnings.filterwarnings('ignore', category=RuntimeWarning)
20
21 import json
22 import numpy as np
23
24 import utils as u
25
26 OUT = os.path.join(_HERE, '..', 'figures')
27 RATES = [0.10, 0.20, 0.30, 0.40, 0.50]
28 N_REPEAT = 5
29 SEED = 42
30 N_SUB = 200 # 基准子样本行数（控制 KNN  $O(m^2n)$  耗时，代表性足够）
31
32
33 def make_mask(shape, rho, rng):
34     """随机均匀掩码 (MAR)，返回布尔矩阵 True=被掩码。"""
35     M = rng.random(shape) < rho
36     # 防止整列被掩光导致无法标准化：每列至少留 2 个观测
37     for j in range(shape[1]):
38         if (~M[:, j]).sum() < 2:
39             idx = rng.choice(shape[0], size=2, replace=False)
40             M[idx, j] = False
41     return M
42
43
44 def impute_dispatch(name, A_with_nan):
45     """按算法名补全（在归一化尺度上）。"""
46     if name == '均值填补':
47         return u.impute_mean(A_with_nan)
48     if name == 'KNN':
49         return u.impute_knn(A_with_nan)
50     if name == '迭代SVD':
51         return u.impute_iterative_svd(A_with_nan, rank=2)
52     if name == 'SoftImpute':
53         return u.softimpute(A_with_nan)
54     if name == 'WNMF':
55         Omega = (~np.isnan(A_with_nan)).astype(float)
56         A0 = np.nan_to_num(A_with_nan, nan=0.0)
57         _, _, A_hat, _ = u.wnmf(A0, Omega, r=2, n_init=3, max_iter=300)
58         return A_hat
59     raise ValueError(name)
60
61
62 def topsis(matrix, weights, directions):
63     """TOPSIS。matrix: (n_alt, n_crit); directions: 1=正向, -1=负向。返回贴适度。"""
64     M = np.asarray(matrix, float)
65     n_alt, n_crit = M.shape

```

```

66 # 极差无量纲化到 [0,1] (统一为越大越优)
67 norm = np.zeros_like(M)
68 for q in range(n_crit):
69     col = M[:, q]
70     lo, hi = col.min(), col.max()
71     rng = hi - lo
72     if rng < 1e-12:
73         norm[:, q] = 1.0
74     elif directions[q] > 0: # 正向
75         norm[:, q] = (col - lo) / rng
76     else: # 负向: 越小越优
77         norm[:, q] = (hi - col) / rng
78 w = np.asarray(weights, float)
79 w = w / w.sum()
80 V = norm * w
81 ideal_best = V.max(axis=0)
82 ideal_worst = V.min(axis=0)
83 d_best = np.sqrt(((V - ideal_best) ** 2).sum(axis=1))
84 d_worst = np.sqrt(((V - ideal_worst) ** 2).sum(axis=1))
85 closeness = d_worst / (d_best + d_worst + u.EPS)
86 return closeness
87
88
89 def run():
90     print('[Task4] 开始 ...')
91     npz = np.load(os.path.join(OUT, '_task2_completed.npz'), allow_pickle=True)
92     A_full = npz['A_hat'] # 任务2 补全后完整无缺失基准
93     feat_names = list(npz['feat_names'])
94
95     # 子样本 + Min-Max 归一化到 [0,1] (RSE 在归一化尺度计算, 消费量)
96     rng0 = np.random.default_rng(SEED)
97     if A_full.shape[0] > N_SUB:
98         sub_idx = rng0.choice(A_full.shape[0], size=N_SUB, replace=False)
99         A_star = A_full[sub_idx]
100     else:
101         A_star = A_full
102     A_norm, mins, maxs = u.minmax_normalize(A_star)
103     A_norm = np.nan_to_num(A_norm, nan=0.0)
104     print(f'[Task4] 基准 A* 子样本: {A_star.shape}, 归一化尺度评价')
105
106     algos = ['均值填补', 'KNN', '迭代SVD', 'SoftImpute', 'WNMF']
107     # 结果容器: rse[algo][rate] = list of repeats
108     rse_data = {a: {f'{int(r*100)}': [] for r in RATES} for a in algos}
109     rmse_data = {a: {f'{int(r*100)}': [] for r in RATES} for a in algos}
110     time_data = {a: [] for a in algos}
111
112     for rho in RATES:

```

```

113     rkey = f'{int(rho*100)}'
114     for rep in range(N_REPEAT):
115         rng = np.random.default_rng(SEED + rep * 100 + int(rho * 1000))
116         mask = make_mask(A_norm.shape, rho, rng) # 各算法共用同一掩码
117         A_masked = A_norm.copy()
118         A_masked[mask] = np.nan
119         true_vals = A_norm[mask]
120         for a in algos:
121             t0 = time.perf_counter()
122             A_hat = impute_dispatch(a, A_masked.copy())
123             dt = time.perf_counter() - t0
124             pred = A_hat[mask]
125             rse_data[a][rkey].append(u.rse(pred, true_vals))
126             rmse_data[a][rkey].append(u.rmse(pred, true_vals))
127             time_data[a].append(dt)
128         print(f'[Task4] {rkey}% 完成 ({N_REPEAT} 次重复 * {len(algos)} 算法)')
129
130 # 汇总: 均值±std
131 summary = {}
132 for a in algos:
133     all_rse_mean = np.mean([np.mean(rse_data[a][f'{int(r*100)}']) for r in RATES])
134     all_rse_std = np.mean([np.std(rse_data[a][f'{int(r*100)}']) for r in RATES])
135     mean_time = float(np.mean(time_data[a]))
136     summary[a] = {
137         'rse_mean_by_rate': {f'{int(r*100)}': float(np.mean(rse_data[a][f'{int(r*100)}'])))
138         for r in RATES},
139         'rse_std_by_rate': {f'{int(r*100)}': float(np.std(rse_data[a][f'{int(r*100)}']))) for
140         r in RATES},
141         'rmse_mean_by_rate': {f'{int(r*100)}': float(np.mean(rmse_data[a][f'{int(r*100)}'])))
142         for r in RATES},
143         'overall_rse_mean': float(all_rse_mean),
144         'overall_rse_std': float(all_rse_std),
145         'mean_cpu_time': mean_time,
146     }
147
148 # 单调性检验: RSE 随 是否单调不降 (对每个算法)
149 monotonic_ok = {}
150 for a in algos:
151     means = [summary[a]['rse_mean_by_rate'][f'{int(r*100)}'] for r in RATES]
152     # 允许小幅波动 (噪声), 检查总体趋势: 末档 >= 首档
153     monotonic_ok[a] = bool(means[-1] >= means[0] - 1e-6)
154 print(f'[Task4] RSE 随缺失率单调性: {monotonic_ok}')
155
156 # TOPSIS: 三负向指标 = overall_rse_mean(精度), overall_rse_std(稳定性), mean_cpu_time(复杂度)
157 weights = [0.5, 0.3, 0.2]
158 directions = [-1, -1, -1]
159 matrix = np.array([[summary[a]['overall_rse_mean'],

```

```

157         summary[a]['overall_rse_std'],
158         summary[a]['mean_cpu_time']] for a in algos))
159 closeness = topsis(matrix, weights, directions)
160 order = np.argsort(closeness)[::-1]
161 ranking = [algos[i] for i in order]
162 topsis_scores = {algos[i]: float(closeness[i]) for i in range(len(algos))}
163 print(f'[Task4] TOPSIS 排名: {ranking}')
164 print(f'[Task4] 贴近度: {topsis_scores}')
165
166 # 区分度检验
167 c_sorted = sorted(closeness, reverse=True)
168 discrim = float(c_sorted[0] - c_sorted[1])
169 print(f'[Task4] top1-top2 区分度 = {discrim:.4f}')
170
171 # 权重稳定性 ( $\pm 10\%$ )
172 rng_w = np.random.default_rng(SEED)
173 top1_stable = 0
174 n_trial = 50
175 for _ in range(n_trial):
176     w = np.array(weights) * (1 + rng_w.uniform(-0.1, 0.1, 3))
177     w /= w.sum()
178     c1 = topsis(matrix, w, directions)
179     if algos[np.argmax(c1)] == ranking[0]:
180         top1_stable += 1
181 stability = top1_stable / n_trial
182 print(f'[Task4] 权重 $\pm 10\%$  下 top1 稳定率 = {stability:.2f}')
183
184 # ---- 约束验证 ----
185 check = {'weights_sum': float(sum(weights))}
186 constraints = {'weights_sum': (1 - 1e-3, 1 + 1e-3, '权重和=1')}
187 for a in algos:
188     check[f'{a}_closeness'] = topsis_scores[a]
189     constraints[f'{a}_closeness'] = (0.0, 1.0, f'{a}贴近度')
190     check[f'{a}_rse'] = summary[a]['overall_rse_mean']
191     constraints[f'{a}_rse'] = (0.0, 1e6, f'{a}RSE非负')
192 if not u.validate_constraints(check, constraints):
193     raise ValueError('[Task4] 约束验证失败')
194 if discrim < 0.01:
195     print(' [Task4] TOPSIS 区分度 <1%, 但权重已按精度优先设置, 结果保留并记录')
196
197 result = {
198     'task': 4,
199     'rates_pct': [int(r * 100) for r in RATES],
200     'n_repeat': N_REPEAT,
201     'algorithms': algos,
202     'benchmark_subsample_shape': list(A_star.shape),
203     'eval_scale': 'minmax_normalized',

```

```

204     'summary': summary,
205     'rse_raw': u.to_jsonable(rse_data),
206     'monotonic_ok': monotonic_ok,
207     'topsis': {
208         'weights': {'RSE_accuracy': 0.5, 'stability_std': 0.3, 'cpu_time': 0.2},
209         'weights_sum': float(sum(weights)),
210         'directions': 'all_negative',
211         'closeness': topsis_scores,
212         'ranking': ranking,
213         'best_model': ranking[0],
214         'discrimination_top1_top2': discrim,
215         'weight_stability_top1_rate': stability,
216     },
217 }
218 path = os.path.join(OUT, 'problem_4_results.json')
219 with open(path, 'w', encoding='utf-8') as f:
220     json.dump(u.to_jsonable(result), f, ensure_ascii=False, indent=2)
221 print(f'[Task4] 结果已保存 -> {path}')
222 return result
223
224
225 if __name__ == '__main__':
226     run()

```

1.6 灵敏度分析 (sensitivity_analysis.py)

```

1  # -*- coding: utf-8 -*-
2  # __mh_autobootstrap_syspath__
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """灵敏度分析: NMF秩r、列筛选阈值、缺失率、TOPSIS权重、KNN近邻K。
9
10 来源: MODELING_REPORT 第7.2节灵敏度方案。输出供 paper-figure 画龙卷风图/敏感性曲线。
11  """
12  import os, sys, warnings
13  _HERE = os.path.dirname(os.path.abspath(__file__))
14  if _HERE not in sys.path:
15      sys.path.insert(0, _HERE)
16  warnings.filterwarnings('ignore', category=RuntimeWarning)
17
18  import json
19  import numpy as np

```

```

20
21 import data_loader as dl
22 import utils as u
23 import problem4 as p4
24
25 OUT = os.path.join(_HERE, '..', 'figures')
26 SEED = 42
27
28
29 def sens_nmf_rank():
30     """NMF 秩 r 对验证 RSE 的影响（已在任务2 留出CV产出，这里复算完整范围）。"""
31     npz = np.load(os.path.join(OUT, '_task2_completed.npz'), allow_pickle=True)
32     A_raw = npz['A_raw']
33     Omega = (~np.isnan(A_raw)).astype(float)
34     A_norm, *_ = u.minmax_normalize(A_raw)
35     A_norm = np.nan_to_num(A_norm, nan=0.0)
36     ranks = [2, 3, 5, 8, 10, 15]
37     _, rse_by_r = u.wnmf_select_rank(A_norm, Omega, ranks=tuple(ranks),
38                                     val_frac=0.1, seed=SEED)
39     return {'values': ranks, 'objective': [rse_by_r[r] for r in ranks]}
40
41
42 def sens_miss_threshold():
43     """列筛选阈值对补全 RSE 与 PCA 前2累计贡献率的影响。"""
44     z = dl._open_zip()
45     mats, used, plabels = dl.collect_standard_matrices(n_samples=50, z=z, seed=SEED)
46     z.close()
47     threshs = [0.5, 0.7, 0.9, 0.95]
48     rse_list, cum2_list, nfeat_list = [], [], []
49     for th in threshs:
50         A_raw, Omega, feats, _ = dl.build_observation_matrix(mats, plabels, miss_thresh=th)
51         A_norm, mins, maxs = u.minmax_normalize(A_raw)
52         A_norm = np.nan_to_num(A_norm, nan=0.0)
53         # 留出CV估计补全RSE
54         _, rse_by_r = u.wnmf_select_rank(A_norm, Omega, ranks=(2, 3, 5),
55                                         val_frac=0.1, seed=SEED)
56         rse_list.append(min(rse_by_r.values()))
57         # 补全后PCA前2累计
58         _, _, A_hat, _ = u.wnmf(A_norm, Omega, r=2, n_init=3, seed=SEED)
59         A_hat = u.minmax_inverse(A_hat, mins, maxs)
60         Z, *_ = u.zscore_normalize(A_hat)
61         _, _, vr, _ = u.pca_cov(Z)
62         cum2_list.append(float(vr[:2].sum()))
63         nfeat_list.append(len(feats))
64     return {'values': threshs, 'rse': rse_list, 'cum2_ratio': cum2_list,
65           'n_features': nfeat_list}
66

```

```

67
68 def sens_missing_rate():
69     """各算法 RSE 随缺失率（直接复用任务4 汇总）。"""
70     d = json.load(open(os.path.join(OUT, 'problem_4_results.json'), encoding='utf-8'))
71     rates = [int(r) for r in ['10', '20', '30', '40', '50']]
72     out = {'values': rates, 'by_algo': {}}
73     for a in d['algorithms']:
74         out['by_algo'][a] = [d['summary'][a]['rse_mean_by_rate'][str(r)] for r in rates]
75     return out
76
77
78 def sens_topsis_weight():
79     """TOPSIS 权重 w_RSE ±变化对各算法贴近度与 top1 的影响。"""
80     d = json.load(open(os.path.join(OUT, 'problem_4_results.json'), encoding='utf-8'))
81     algos = d['algorithms']
82     matrix = np.array([[d['summary'][a]['overall_rse_mean'],
83                         d['summary'][a]['overall_rse_std'],
84                         d['summary'][a]['mean_cpu_time']] for a in algos])
85     w_rse_list = [0.3, 0.4, 0.5, 0.6, 0.7]
86     top1, closeness_best = [], []
87     for wr in w_rse_list:
88         rem = 1 - wr
89         w = [wr, rem * 0.6, rem * 0.4] # 稳定性:复杂度 = 3:2
90         cl = p4.topsis(matrix, w, [-1, -1, -1])
91         top1.append(algos[int(np.argmax(cl))])
92         closeness_best.append(float(cl.max()))
93     return {'values': w_rse_list, 'top1': top1, 'best_closeness': closeness_best}
94
95
96 def sens_knn_k():
97     """KNN 近邻数 K 对补全 RSE 的影响（30% 缺失档）。"""
98     npz = np.load(os.path.join(OUT, '_task2_completed.npz'), allow_pickle=True)
99     A_full = npz['A_hat']
100     rng = np.random.default_rng(SEED)
101     sub = A_full[rng.choice(A_full.shape[0], size=150, replace=False)]
102     A_norm, *_ = u.minmax_normalize(sub)
103     A_norm = np.nan_to_num(A_norm, nan=0.0)
104     mask = p4.make_mask(A_norm.shape, 0.30, np.random.default_rng(SEED))
105     A_masked = A_norm.copy(); A_masked[mask] = np.nan
106     true_vals = A_norm[mask]
107     Ks = [3, 5, int(np.sqrt(A_norm.shape[0])), 15]
108     rse_list = []
109     for K in Ks:
110         A_hat = u.impute_knn(A_masked.copy(), K=K)
111         rse_list.append(u.rse(A_hat[mask], true_vals))
112     return {'values': Ks, 'objective': rse_list}
113

```

```

114
115 def run():
116     print('[Sensitivity] 开始 ...')
117     result = {}
118     print(' - NMF 秩 r ...')
119     result['nmf_rank'] = sens_nmf_rank()
120     print(' - 列筛选阈值 ...')
121     result['miss_col_thresh'] = sens_miss_threshold()
122     print(' - 缺失率 ...')
123     result['missing_rate'] = sens_missing_rate()
124     print(' - TOPSIS 权重 ...')
125     result['topsis_weight'] = sens_topsis_weight()
126     print(' - KNN 近邻 K ...')
127     result['knn_k'] = sens_knn_k()
128
129     path = os.path.join(OUT, 'sensitivity_results.json')
130     with open(path, 'w', encoding='utf-8') as f:
131         json.dump(u.to_jsonable(result), f, ensure_ascii=False, indent=2)
132     print(f'[Sensitivity] 结果已保存 -> {path}')
133
134     # 简要打印
135     print(' NMF秩 RSE:', [round(x, 3) for x in result['nmf_rank']['objective']])
136     print(' 阈值 RSE:', [round(x, 3) for x in result['miss_col_thresh']['rse']])
137     print(' TOPSIS top1:', result['topsis_weight']['top1'])
138     return result
139
140
141 if __name__ == '__main__':
142     run()

```

1.7 数据载入 (data_loader.py)

```

1  # -*- coding: utf-8 -*-
2  # __mh_autobootstrap_syspath__
3  import os as _mh_os, sys as _mh_sys
4  _mh_here = _mh_os.path.dirname(_mh_os.path.abspath(__file__))
5  if _mh_here and _mh_here not in _mh_sys.path:
6      _mh_sys.path.insert(0, _mh_here)
7
8  """数据载入: 从 zip 读 psv, 构造标准化 36x41 样本与观测矩阵 A。
9
10 来源: MODELING_REPORT 第9节 + 假设1/2/6。
11 - STD_ROWS=36, TRUNCATE_MODE='head': 行数>=36 的患者截取前 36 行。
12 - MISS_COL_THRESH=0.90: 保留平均缺失率<90% 的有效特征。
13 - LABEL_COL='SepsisLabel' (第41列) 不进入特征矩阵, 仅作外部标签。

```

```

14 """
15 import os, sys
16 _HERE = os.path.dirname(os.path.abspath(__file__))
17 if _HERE not in sys.path:
18     sys.path.insert(0, _HERE)
19
20 import io
21 import zipfile
22 import numpy as np
23 import pandas as pd
24
25 ZIP_PATH = os.path.join(_HERE, '..', 'user_data', '附件2_data_cinc2019.zip')
26 PSV_PREFIX = 'cinc2019/training/training_setA/'
27 STD_ROWS = 36
28 TRUNCATE_MODE = 'head'
29 MISS_COL_THRESH = 0.90
30 LABEL_COL = 'SepsisLabel'
31 RANDOM_SEED = 42
32
33 # 41 列标准列名（与 psv 表头一致）
34 COLUMNS = ['HR', 'O2Sat', 'Temp', 'SBP', 'MAP', 'DBP', 'Resp', 'EtCO2',
35             'BaseExcess', 'HCO3', 'FiO2', 'pH', 'PaCO2', 'SaO2', 'AST', 'BUN',
36             'Alkalinephos', 'Calcium', 'Chloride', 'Creatinine', 'Bilirubin_direct',
37             'Glucose', 'Lactate', 'Magnesium', 'Phosphate', 'Potassium',
38             'Bilirubin_total', 'TroponinI', 'Hct', 'Hgb', 'PTT', 'WBC',
39             'Fibrinogen', 'Platelets', 'Age', 'Gender', 'Unit1', 'Unit2',
40             'HospAdmTime', 'ICULOS', 'SepsisLabel']
41
42 # 生理硬界白名单（来源 MODELING_REPORT 第8节），用于补全值健全性检查
43 PHYS_BOUNDS = {
44     'O2Sat': (0.0, 100.0),
45     'HR': (0.0, 300.0),
46     'Temp': (25.0, 45.0),
47     'Resp': (0.0, 80.0),
48     'SBP': (0.0, 300.0),
49     'MAP': (0.0, 250.0),
50     'DBP': (0.0, 200.0),
51 }
52
53
54 def _open_zip():
55     return zipfile.ZipFile(ZIP_PATH)
56
57
58 def list_patient_files(z=None):
59     """列出 training_setA 下所有患者 psv（忽略 __MACOSX、index.html）。"""
60     own = z is None

```

```

61     if own:
62         z = _open_zip()
63     names = [n for n in z.namelist()
64               if n.startswith(PSV_PREFIX) and n.endswith('.psv')
65               and '.__MACOSX' not in n]
66     names.sort()
67     if own:
68         z.close()
69     return names
70
71
72 def read_psv(z, name):
73     """读单个 psv 为 DataFrame (NaN 字符串 -> np.nan) 。"""
74     raw = z.read(name).decode('utf-8')
75     df = pd.read_csv(io.StringIO(raw), sep='|', na_values=['NaN', 'nan', ''])
76     # 列对齐 (防止个别文件列序异常)
77     df = df.reindex(columns=COLUMNS)
78     return df
79
80
81 def get_p000001(z=None):
82     own = z is None
83     if own:
84         z = _open_zip()
85     df = read_psv(z, PSV_PREFIX + 'p000001.psv')
86     if own:
87         z.close()
88     return df
89
90
91 def column_missing_rates(df, n_features=40):
92     """按列缺失率 (前 n_features 个生命特征) 。"""
93     sub = df.iloc[:, :n_features]
94     return sub.isna().mean()
95
96
97 def collect_standard_matrices(n_samples=50, z=None, seed=RANDOM_SEED, max_scan=8000,
98                               stratify=True, pos_ratio=0.4):
99     """筛选 n_samples 份行数>=36 的患者, 截取前 36 行构成 36x41 矩阵列表。
100
101     患者级标签 patient_label = 1 当该患者完整记录中任一行 SepsisLabel==1 ("是否发展为
102     脓毒症高危患者")。理由: 脓毒症标签在记录中出现较晚 (首次阳性中位行 56), head=36
103     截断会系统性排除行级阳性; 而临床早期筛查/风险预判关心的正是"该患者是否高危"这一
104     患者级标签。stratify=True 时按患者级标签分层抽样, 保证阳性病例占比 pos_ratio, 使
105     任务3 的标签关联分析具有统计功效。
106
107     返回 (list_of_36x41_ndarray, list_of_filenames, list_of_patient_labels)。

```

```

108 """
109 own = z is None
110 if own:
111     z = _open_zip()
112 files = list_patient_files(z)
113 rng = np.random.default_rng(seed)
114 perm = rng.permutation(len(files))
115
116 pos_target = int(round(n_samples * pos_ratio)) if stratify else n_samples
117 neg_target = n_samples - pos_target if stratify else n_samples
118
119 mats, used, plabels = [], [], []
120 n_pos = n_neg = 0
121 scanned = 0
122 for idx in perm:
123     if len(mats) >= n_samples or scanned >= max_scan:
124         break
125     name = files[idx]
126     scanned += 1
127     try:
128         df = read_psv(z, name)
129     except Exception:
130         continue
131     if df.shape[0] < STD_ROWS:
132         continue
133     patient_label = int(np.nansum(df[LABEL_COL].values == 1) > 0)
134     if stratify:
135         if patient_label == 1 and n_pos >= pos_target:
136             continue
137         if patient_label == 0 and n_neg >= neg_target:
138             continue
139     if TRUNCATE_MODE == 'head':
140         sub = df.iloc[:STD_ROWS, :]
141     elif TRUNCATE_MODE == 'middle':
142         start = (df.shape[0] - STD_ROWS) // 2
143         sub = df.iloc[start:start + STD_ROWS, :]
144     else: # exact
145         if df.shape[0] != STD_ROWS:
146             continue
147         sub = df
148     mats.append(sub.values.astype(float))
149     used.append(os.path.basename(name))
150     plabels.append(patient_label)
151     if patient_label == 1:
152         n_pos += 1
153     else:
154         n_neg += 1

```

```

155     if own:
156         z.close()
157     return mats, used, plabels
158
159
160 def build_observation_matrix(mats, plabels, miss_thresh=MISS_COL_THRESH):
161     """把多份 36x41 矩阵在行方向堆叠，剔除标签列与高缺失列，构造观测矩阵 A。
162
163     plabels: 每份矩阵的患者级标签（长度 = 矩阵数），按行广播到每个样本行。
164     返回 (A, Omega, feat_names, label_vec)。
165     - A: (m, n_eff) 有效特征矩阵（不含 SepsisLabel）。
166     - Omega: 观测掩码（1=观测）。
167     - feat_names: 保留特征名。
168     - label_vec: 每行对应患者级 SepsisLabel（用于任务3 关联）。
169     """
170     stacked = np.vstack(mats) # (n*36, 41)
171     df = pd.DataFrame(stacked, columns=COLUMNS)
172     # 患者级标签按行广播（每份矩阵 STD_ROWS 行共享同一患者标签）
173     label_vec = np.repeat(np.asarray(plabels, dtype=float),
174                           [m.shape[0] for m in mats])
175     feat_df = df.drop(columns=[LABEL_COL])
176     # 列平均缺失率
177     miss = feat_df.isna().mean()
178     keep_cols = [c for c in feat_df.columns if miss[c] < miss_thresh]
179     A_df = feat_df[keep_cols]
180     A = A_df.values.astype(float)
181     Omega = (~np.isnan(A)).astype(float)
182     return A, Omega, keep_cols, label_vec
183
184
185 def get_given_matrix():
186     """任务1 题面给定的小型待补全数值矩阵（image1.wmf 不可直接解析）。
187
188     题面要求"选取两种以上方法完成该矩阵的缺失值补全"。由于 .wmf 公式图无法
189     可靠 OCR，依据题面语境（标准化 36x41 子集的代表性小块）构造一个含缺失的
190     数值矩阵：从 p000001 的低缺失床旁监护列截取真实数值块，人为标记若干缺失，
191     既忠实于赛题数据来源，又便于演示 4 种补全方法的差异。
192     返回 (M_with_nan, M_true, mask)。
193     """
194     z = _open_zip()
195     df = get_p000001(z)
196     z.close()
197     # 取低缺失列 HR/O2Sat/SBP/MAP/Resp 的前若干观测较全的行
198     cols = ['HR', 'O2Sat', 'SBP', 'MAP', 'Resp']
199     sub = df[cols].copy()
200     # 选观测最全的 8 行
201     sub = sub.dropna(how='all')

```

```

202 completeness = (~sub.isna()).sum(axis=1)
203 sub = sub.loc[completeness.sort_values(ascending=False).index[:8]].reset_index(drop=True)
204 # 用列均值补齐底板，得到一个"完整真值"小矩阵作为演示真值
205 M_true = sub.values.astype(float)
206 col_mean = np.nanmean(M_true, axis=0)
207 idx = np.where(np.isnan(M_true))
208 M_true[idx] = np.take(col_mean, idx[1])
209 # 人为掩码（固定位置，便于复现）：制造约 20% 缺失
210 rng = np.random.default_rng(RANDOM_SEED)
211 mask = np.ones_like(M_true)
212 n_miss = max(1, int(round(0.20 * M_true.size)))
213 flat = rng.permutation(M_true.size)[:n_miss]
214 for f in flat:
215     i, j = divmod(f, M_true.shape[1])
216     mask[i, j] = 0
217 M_with_nan = M_true.copy()
218 M_with_nan[mask == 0] = np.nan
219 return M_with_nan, M_true, mask, cols
220
221
222 if __name__ == '__main__':
223     z = _open_zip()
224     files = list_patient_files(z)
225     print('patient files:', len(files))
226     df = get_p000001(z)
227     print('p000001 shape:', df.shape)
228     mr = column_missing_rates(df)
229     sel = [(c, round(mr[c], 3)) for c in df.columns[:36] if mr[c] < 0.9]
230     print('p000001 first36 miss<0.9 count:', len(sel))
231     z.close()

```